CopernicusLAC Infrastructure Support

Deployment Guide

Advancing Earth Science

Title	Copernicus LAC Infra - Deployment Guide
Prepared by	Terradue
Reference	T2-ESA-COPLACI-PR-25-007
Issue	2.2
Date	2026-10-23
Status	Issued
Document Type	Procedure
Configuration	Configured document
ESA Contract No:	4000144570/24/I-KE

Change record

Date	Issue	Section	Page	Comment
2025-03-25	1.0	All	All	Initial issue focusing on prerequisites and the Data Discovery components
2025-05-06	1.1	5	22 - 24	Added detailed installation steps for TiTiler-pgSTAC
2025-05-21	1.2	6	25 - 27	Added detailed installation steps for STAC Browser
2025-07-27	1.3	All	All	Separated prerequisites from components deployment; added installation steps for additional components; added information about deployment automation
2025-12-28	1.4	4.9 4.10 6	52 - 64 64 - 66 75 - 78	Front-end services User Workspace Deployment status
2026-10-01	2.0	All	All	Complete rework for the final architecture: GitOps with Argo CD and Crossplane v2 as the only delivery path; deployment tiers (Argo CD and GitLab outside Crossplane); infrastructure, node pools, Longhorn, Contour, managed PostgreSQL 16 HA and StorageGRID object storage outside the cluster; all components including processing API, CWL runner, application packages, events, Portal, App Hub, Drive; operations, known issues and migration plan. OpenEBS, NGINX Ingress, Kafka UI and provider-argocd removed. See Annex B.
2026-10-15	2.1	All	All	Deployment of the Panama platform with the final delivery model: tiers redefined (Tier 1 bootstrap services as Argo CD Applications with manual sync, Tier 2 Crossplane with helmfile, Tier 3 layers 10–70); repository structure, delivery package, SOPS/age secrets, bootstrap.sh, Harbor and Harbor seed; actual versions and procedures of every deployed component; security model (OPA, oauth2-proxy, route pattern); deployment status, open points and temporary settings. See Annex B.
2026-10-23	2.2	All	All	Tier 1 published (DNS git. and cr.): exposure, Harbor seed, GitLab and Harbor sign-in through Keycloak, Harbor robot accounts (harbor-robots, harbor-oidc); Crossplane packages and all charts served from Harbor; Git access for operators; PostgreSQL shared_buffers applied; StorageGRID 12.0.0 issue (403 instead of 404 on missing objects) blocking image pushes to Harbor; deployment status, open points and temporary settings updated.

© Terradue 2026 - The copyright in this document is vested in Terradue. This document may only be reproduced in whole or in part, or stored in a retrieval system, or transmitted in any form, or by any means electronic, mechanical, photocopying or otherwise, either with the prior permission of Terradue or in accordance with the terms of ESA Contract No. 4000144570/24/I-KE.

Table of Content

1 Introduction **5**
1.1 Purpose 5
1.2 Background 5
1.3 Scope 5
1.4 Intended Audience and Use in Training 6
1.5 Document Structure 6
1.6 Conventions 7
1.7 Applicable Documents 7
1.8 Reference Documents 7
1.9 Acronyms and Abbreviations 8
2 Platform Architecture **10**
2.1 Functional View 10
2.2 Logical Layers 11
2.3 Platform at a Glance 11
2.4 Deployment Tiers and Ownership 12
2.5 Component Inventory 13
2.6 Key Design Decisions 15
3 Infrastructure **16**
3.1 Kubernetes Cluster 16
3.2 Node Pools 16
3.3 Storage 17
3.4 Network, DNS and Ingress 17
3.5 Managed Data Services 18
3.5.1 PostgreSQL 18
3.5.2 Object Storage 20
3.6 Other External Dependencies 20
3.7 Access and Tooling 21
4 Delivery Package and Repository **22**
4.1 Principles 22
4.2 The Delivery Package 22
4.3 Repository Structure 23
4.4 Secrets Model 24
4.5 End-to-end Sequence 25
5 Tiers 1 and 2 — Bootstrap **27**
5.1 bootstrap.sh 27
5.2 Argo CD 28
5.3 cert-manager and Contour 28
5.4 Valkey and GitLab 29
5.5 Harbor 30
5.6 Exposure of Tier 1 Services 31
5.7 Harbor Seed 31
5.8 Crossplane (Tier 2) 32
6 Tier 3 — Platform Delivery **33**
6.1 Root Application and Layers 33
6.2 Authoring Conventions 33
6.3 provider-helm Behaviour 35
6.4 Adding a Component 35

© Terradue 2026 - The copyright in this document is vested in Terradue. This document may only be reproduced in whole or in part, or stored in a retrieval system, or transmitted in any form, or by any means electronic, mechanical, photocopying or otherwise, either with the prior permission of Terradue or in accordance with the terms of ESA Contract No. 4000144570/24/I-KE.

7 Security and Platform Foundation **37**
7.1 Vault 37
7.2 External Secrets Operator 38
7.3 TLS and Exposure Pattern 38
7.4 Keycloak 39
7.5 OPA 40
7.6 Protecting a Public Service 41
7.7 Knative Serving 42
8 Data Services **43**
8.1 Kafka 43
8.2 Kafka UI 44
8.3 STAC API 44
8.4 TiTiler-pgSTAC (to deploy) 45
8.5 Geobrowser (to deploy) 45
8.6 STAC Browser (to deploy) 46
9 Processing Services **47**
9.1 Argo Workflows 47
9.2 Argo Events, Event Bus and Webhook 48
9.3 Processing API (mate-api) 49
9.4 CWL Runner and Application Packages (to deploy) 49
10 User-facing Services **50**
10.1 Portal, App Hub and Drive (to deploy) 50
11 Operations **51**
11.1 Shipping a Change 51
11.2 Daily Health Check 51
11.3 Troubleshooting 52
11.4 Backup and Recovery - TBV by IFX 54
11.5 Disaster Recovery - TBV by IFX 54
12 Deployment Status and Open Points **55**
12.1 Deployment Status 55
12.2 Open Points 56
12.3 Temporary Settings to Revert 56
13 Support and Contacts **57**
Annex A – Public Endpoints **58**
Annex B – Changes with Respect to Issue 2.1 **59**
Annex C – Vault Paths and ExternalSecrets **60**

© Terradue 2026 - The copyright in this document is vested in Terradue. This document may only be reproduced in whole or in part, or stored in a retrieval system, or transmitted in any form, or by any means electronic, mechanical, photocopying or otherwise, either with the prior permission of Terradue or in accordance with the terms of ESA Contract No. 4000144570/24/I-KE.

1 Introduction

1.1 Purpose

This document provides the technical reference for deploying, configuring and operating the CopernicusLAC platform in Panama. It describes the platform architecture, infrastructure prerequisites, deployment tiers, delivery repository and procedures for installing and verifying platform components.

The guide is intended for engineers responsible for deployment, operations and technical handover, and will support the associated training activities. It will be used together with the delivery repository, which contains the manifests, configuration files and operational scripts.

This is a living document that will be updated and completed over the coming months as part of the platform's operational readiness preparation. Updates will incorporate completed deployments, validated operational and recovery procedures, and findings from training and hands-on activities.

The document distinguishes deployed components from those awaiting deployment or validation. Section 12 records deployment status, open points and temporary settings. Procedures identified as incomplete or pending confirmation will require validation before operational use.

1.2 Background

The CopernicusLAC initiative is a collaborative effort between the European Space Agency (ESA), the European Commission's Directorate-General for International Partnerships (DG-INTPA), and the Government of Panama. The initiative aims to establish a Regional Copernicus Centre in Panama to facilitate the uptake and utilisation of Copernicus data in the LAC region. The primary focus of the Centre is to support disaster risk reduction and recovery efforts, utilising EO data and geospatial technology to deliver information services for local and regional stakeholders.

This initiative is part of the broader EU-LAC Digital Alliance, which promotes cooperation in digital and space technologies between the European Union (EU) and LAC countries. The CopernicusLAC Centre is envisioned as a pilot project under the EU's Global Gateway strategy, aiming to foster international collaboration, enhance data sharing, and support the development of local capacities in EO data processing and utilisation. Once established, the CopernicusLAC Centre, hosted in Panama, will serve as a regional hub for Earth Observation (EO) data, providing essential services tailored to the unique challenges and needs of the LAC region.

1.3 Scope

This document covers:

- the platform architecture: functional view, logical layers, deployment tiers and component inventory;
- the infrastructure and the external prerequisites: RKE2 cluster, node pools, Longhorn, DNS, load balancer, managed PostgreSQL 16 HA and StorageGRID object storage;
- the delivery package handed to the operations team: the infrastructure repository, its structure, the secrets model and the bootstrap tooling;
- the bootstrap of Tiers 1 and 2 (Argo CD, cert-manager, Contour, Valkey, GitLab, Harbor, Crossplane) and the delivery of Tier 3 (layers 10 to 70) through Argo CD and Crossplane;
- the deployment of every platform component, with the exact versions, prerequisites and verification steps;
- the security model of the public services (Keycloak, OPA, oauth2-proxy, routing rules);
- day-2 operations: change workflow, health checks, troubleshooting, upgrades, backup and recovery;
- the deployment status, the open points, the temporary settings to revert and the components still to deploy.

Out of scope: the installation of Rancher and of the virtual machines, the operation of the managed PostgreSQL and object storage services, the DNS zone and the external load balancer, and the development of the processing applications.

© Terradue 2026 - The copyright in this document is vested in Terradue. This document may only be reproduced in whole or in part, or stored in a retrieval system, or transmitted in any form, or by any means electronic, mechanical, photocopying or otherwise, either with the prior permission of Terradue or in accordance with the terms of ESA Contract No. 4000144570/24/I-KE.

1.4 Intended Audience and Use in Training

This document is intended for AIG system administrators and platform engineers, Terradue personnel and infrastructure-provider staff involved in deployment, operations and technical handover.

It will be used alongside the training plan [RD-2], the module-specific training materials and the delivery package described in Section 4.2. Training materials will introduce the concepts and guide practical activities; the repository will provide the associated manifests, configuration files and scripts; and this guide will describe deployment sequences, prerequisites, procedures and verification steps.

During training, the relevant sections will be identified for each module. Findings from practical activities will support the clarification, validation and completion of the guide as part of the platform's operational readiness preparation.

Chapters 2 to 6 provide the architecture, prerequisites and deployment framework and should be reviewed before undertaking deployment activities. Chapters 7 to 10 describe individual platform components, Chapter 11 covers operational procedures, and Chapter 12 identifies deployment status and outstanding actions. Procedures marked as incomplete or pending confirmation will require validation before operational use.

1.5 Document Structure

The document is organised as follows:

- Chapter 1 – Introduction: purpose, scope, intended audience, conventions and reference documents.
- Chapter 2 – Platform Architecture: functional view, logical layers, deployment tiers, component inventory and design decisions.
- Chapter 3 – Infrastructure: Kubernetes cluster, node pools, storage, networking, managed data services and operator access requirements.
- Chapter 4 – Delivery Package and Repository: delivery principles, package contents, repository structure, secrets management and deployment sequence.
- Chapter 5 – Tiers 1 and 2 — Bootstrap: installation and configuration of the bootstrap services, Harbor registry and Crossplane.
- Chapter 6 – Tier 3 — Platform Delivery: delivery through Argo CD and Crossplane, configuration conventions and procedures for adding components.
- Chapter 7 – Security and Platform Foundation: Vault, External Secrets Operator, TLS, Keycloak, OPA, public-service protection and Knative.
- Chapter 8 – Data Services: Kafka, Kafka UI, STAC API, TiTiler, Geobrowser and STAC Browser.
- Chapter 9 – Processing Services: Argo Workflows, event handling, the processing API, CWL runner and application packages.
- Chapter 10 – User-facing Services: Portal, App Hub and Drive, including deployment prerequisites and configuration requirements.
- Chapter 11 – Operations: change management, health checks and troubleshooting, together with backup and recovery procedures to be completed and validated.
- Chapter 12 – Deployment Status and Open Points: component status, outstanding actions and temporary settings to be replaced.
- Chapter 13 – Support and Contacts: technical support contact information.
- Annex A – Public Endpoints: service addresses, protection mechanisms and deployment status.
- Annex B – Changes with Respect to Issue 2.0: summary of changes to the architecture and deployment approach.
- Annex C – Vault Paths and ExternalSecrets: inventory of secret locations and their use by platform components.

© Terradue 2026 - The copyright in this document is vested in Terradue. This document may only be reproduced in whole or in part, or stored in a retrieval system, or transmitted in any form, or by any means electronic, mechanical, photocopying or otherwise, either with the prior permission of Terradue or in accordance with the terms of ESA Contract No. 4000144570/24/I-KE.

1.6 Conventions

Table 1 – Conventions used in this document

Convention	Meaning
TEMPORARY	A setting that is in place only until an external prerequisite is met (images not yet in Harbor, Keycloak clients). Each one is marked in the repository with the same word and listed in section 12.3.
TODAY / TARGET	Where the running platform differs from the target, the difference is stated explicitly.
DONE WHEN boxes	The checks that close a step. A step is complete only when every check passes.
<placeholder>	A value supplied by the operator (passwords are never written in commands or in Git).
[TBC-n]	A point to be confirmed; all of them are listed in section 12.2.
Commands	Run from an operator workstation (macOS or Linux) at the root of the repository, with the kubeconfig context copernicuslac-panama obtained from Rancher. Commands that read a secret are written so that the value never reaches the shell history (read -s, stdin, files that are deleted).
Shell	Interactive prompts (read -p) are written for bash. On macOS the default shell is zsh: run them inside bash -c '…' as shown, or in a bash session.
Resource names	Real names from the Panama cluster and repository are used, so that what operators read matches what they see.

1.7 Applicable Documents

Table 2 – Applicable documents

Ref.	Title and References	Issue
[SOW]	CopernicusLAC Infrastructure Support (EOP-EOP-SI-SOW-2023-15)	1.0
[CTR]	ESA Contract No. 4000144570/24/I-KE with Terradue Srl COPERNICUSLAC - INFRASTRUCTURE SUPPORT	01/03/2023
[D1]	CopernicusLAC Centre Specification and Performance Requirements	1.4
[D2]	System Architecture Description	1.3
[D3]	Interface Control Document	1.3
[D11]	Data Exploration and Processing Validation Plan and Report	1.3

1.8 Reference Documents

Table 3 – Reference documents

Ref.	Title and References	Issue
[PROP]	T2-ESA-AO1-11947-OF-23-062 Terradue Offer for CopLAC Infra (all Volumes)	1.0
[RD-2]	T2-ESA-COPLACI-PL-26-049 CopernicusLAC Infra Training Plan	2.4
[RD-3]	Crossplane documentation, v2 — docs.crossplane.io	2.4
[RD-4]	Argo CD documentation — argo-cd.readthedocs.io	3.5
[RD-5]	Longhorn documentation — longhorn.io/docs	1.10
[RD-6]	Strimzi documentation — strimzi.io/documentation	1.1
[RD-7]	Infrastructure repository infra-copernicuslac-panama (delivery package, section 4.2)	2026-10-23

© Terradue 2026 - The copyright in this document is vested in Terradue. This document may only be reproduced in whole or in part, or stored in a retrieval system, or transmitted in any form, or by any means electronic, mechanical, photocopying or otherwise, either with the prior permission of Terradue or in accordance with the terms of ESA Contract No. 4000144570/24/I-KE.

Ref.	Title and References	Issue
[RD-8]	pgSTAC — github.com/stac-utils/pgstac, tag v0.9.12	0.9.12

1.9 Acronyms and Abbreviations

Table 4 – Acronyms and abbreviations

Acronym	Definition
AIG	Autoridad Nacional para la Innovación Gubernamental
API	Application Programming Interface
CDSE	Copernicus Data Space Ecosystem
CNI	Container Network Interface
COG	Cloud Optimized GeoTIFF
CRD	Custom Resource Definition
CWL	Common Workflow Language
DR	Disaster Recovery
EO	Earth Observation
ESA	European Space Agency
ESO	External Secrets Operator
EU	European Union
HA	High Availability
HSTS	HTTP Strict Transport Security
JWT	JSON Web Token
KRaft	Kafka Raft metadata mode (Kafka without ZooKeeper)
KV	Key/Value secrets engine of Vault
LAC	Latin America and the Caribbean
MR	Managed Resource (Crossplane)
OCI	Open Container Initiative (image and artifact format)
OGC	Open Geospatial Consortium
OIDC	OpenID Connect
OPA	Open Policy Agent
OTP	One-Time Password
PVC	Persistent Volume Claim
RBAC	Role-Based Access Control
RKE2	Rancher Kubernetes Engine 2
RWO / RWX	ReadWriteOnce / ReadWriteMany volume access modes
S3	Amazon S3-compatible object storage API
SOPS	Secrets OPerationS — encrypted files in Git (with age keys)
SSO	Single Sign-On
STAC	SpatioTemporal Asset Catalog

© Terradue 2026 - The copyright in this document is vested in Terradue. This document may only be reproduced in whole or in part, or stored in a retrieval system, or transmitted in any form, or by any means electronic, mechanical, photocopying or otherwise, either with the prior permission of Terradue or in accordance with the terms of ESA Contract No. 4000144570/24/I-KE.

Acronym	Definition
TBC	To Be Confirmed
TLS	Transport Layer Security
XR / XRD	Composite Resource / Composite Resource Definition (Crossplane)

© Terradue 2026 - The copyright in this document is vested in Terradue. This document may only be reproduced in whole or in part, or stored in a retrieval system, or transmitted in any form, or by any means electronic, mechanical, photocopying or otherwise, either with the prior permission of Terradue or in accordance with the terms of ESA Contract No. 4000144570/24/I-KE.

2 Platform Architecture

2.1 Functional View

The functional decomposition of the CopernicusLAC Centre platform is unchanged with respect to [D2]. The table maps each functional module to the components that implement it in Panama.

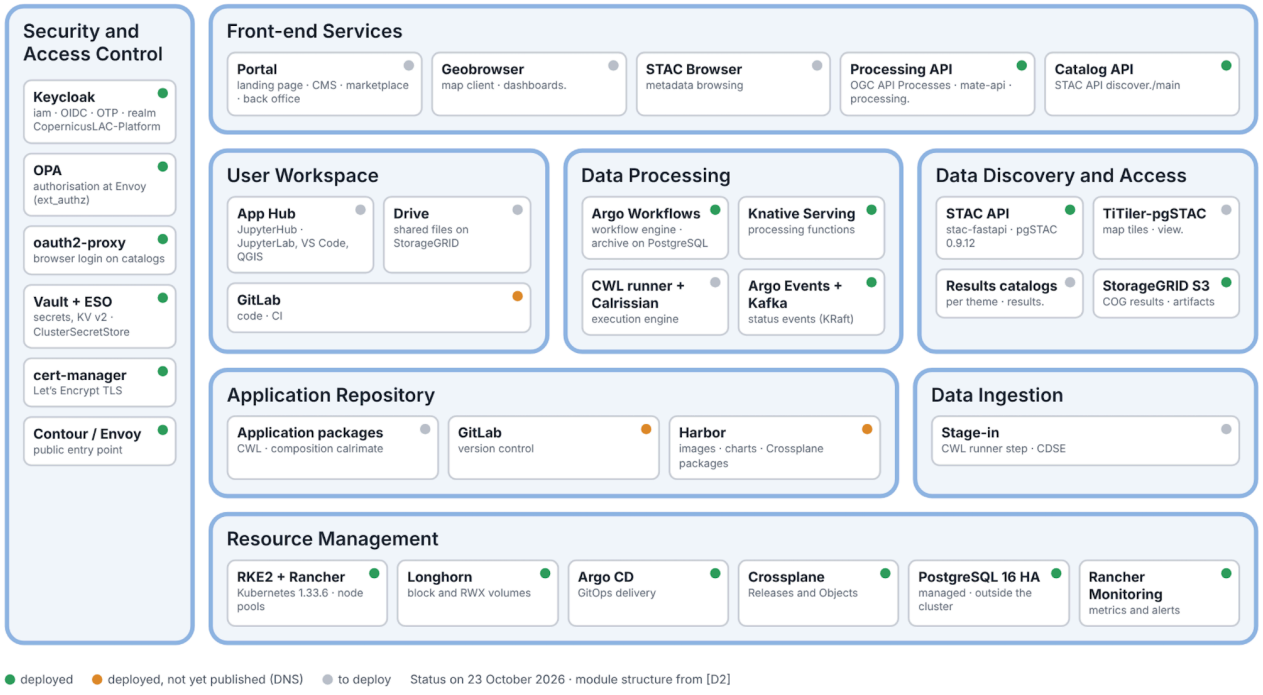


Figure 1 – CopernicusLAC Centre platform functional modules (from [D2])

Table 5 – Functional modules and implementing components

Functional module	Implemented by
Security and Access Control	Keycloak (identity, OIDC, OTP), Vault + External Secrets (secrets), OPA (authorisation at Envoy), oauth2-proxy (browser login on catalogs), cert-manager (TLS)
Data Ingestion	Stage-in step of the CWL runner, reading from the Copernicus Data Space Ecosystem (CDSE)
Data Processing	Processing API (OGC API – Processes, mate-api on Knative), Argo Workflows, CWL runner templates, application packages
Data Discovery and Access	STAC API (stac-fastapi on pgSTAC), TiTiler-pgSTAC, Geobrowser, STAC Browser, StorageGRID object storage
Front-End Services	Portal (landing page, CMS, marketplace, back office)
User Workspace	App Hub (JupyterHub), Drive (shared files), GitLab
Application Repository	GitLab (code), Harbor (container images, charts, Crossplane packages)
Resource Management	RKE2 cluster managed by Rancher, Longhorn, Argo CD, Crossplane, Rancher Monitoring
Messaging	Kafka (Strimzi, KRaft) and Argo Events, carrying processing status events

© Terradue 2026 - The copyright in this document is vested in Terradue. This document may only be reproduced in whole or in part, or stored in a retrieval system, or transmitted in any form, or by any means electronic, mechanical, photocopying or otherwise, either with the prior permission of Terradue or in accordance with the terms of ESA Contract No. 4000144570/24/I-KE.

2.2 Logical Layers

The platform is organised in layers (Figure 2). Each layer depends only on the layers below it. The same ordering drives the bootstrap order (section 4.5), the Argo CD sync waves of Tier 3 (section 6.1) and the troubleshooting order (section 11.3). The folders of platform/ in the repository carry the layer numbers (10-core … 70-application-packages).

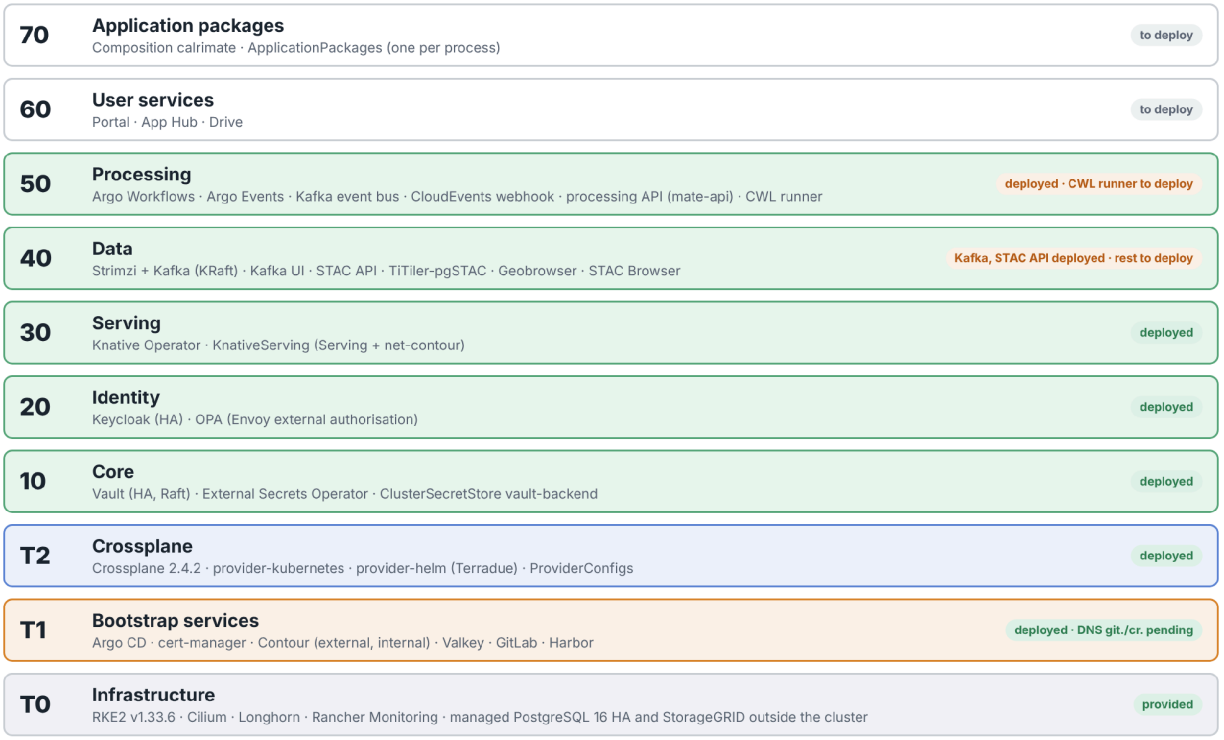


Figure 2 – Logical layers and deployment status (23 October 2026)

2.3 Platform at a Glance

Figure 3 shows the runtime components and how they connect. All public traffic enters through Envoy (Contour); processing is driven by the processing API and executed by Argo Workflows; status events flow to Kafka; results are written to object storage and indexed in the STAC catalog. All relational databases and all bulk data live in two managed services outside the cluster: PostgreSQL 16 in high availability and StorageGRID S3 (section 3.5). The figure is functional: the deployment status of each component is in section 12.1, and the platform also includes Harbor (registry) and Valkey (Redis-compatible caches).

© Terradue 2026 - The copyright in this document is vested in Terradue. This document may only be reproduced in whole or in part, or stored in a retrieval system, or transmitted in any form, or by any means electronic, mechanical, photocopying or otherwise, either with the prior permission of Terradue or in accordance with the terms of ESA Contract No. 4000144570/24/I-KE.

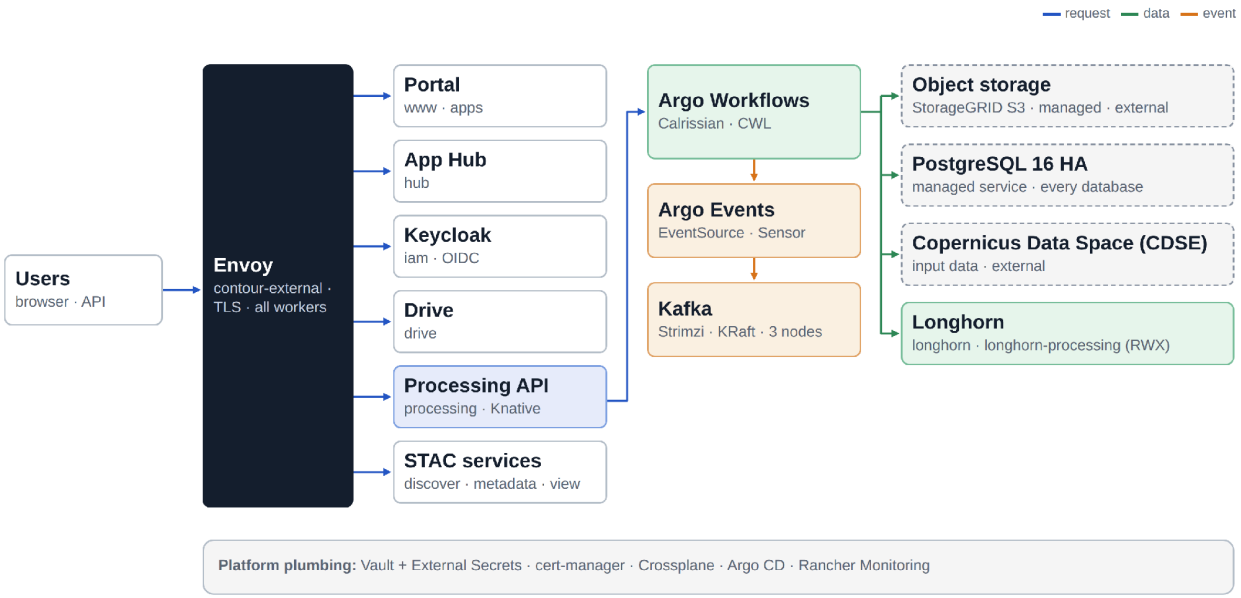
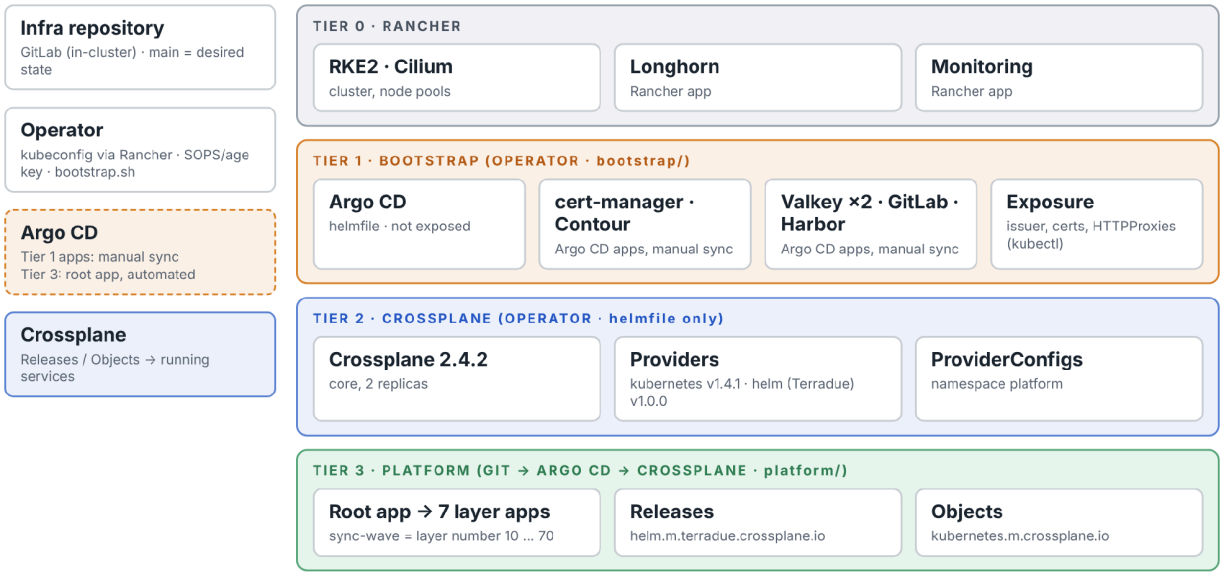


Figure 3 – Platform components and their main connections (functional)

2.4 Deployment Tiers and Ownership

Every object in the cluster has exactly one owner, and every owner sits in a tier below the objects it manages (Figure 4). This removes the circular dependencies that would otherwise make the platform impossible to rebuild or repair: the tools that deliver the platform (Argo CD, GitLab, Harbor, Crossplane) are never delivered by themselves.



Rule: nothing is managed by something it manages. Tiers 1–2 are applied by the operator from *bootstrap/* and are never synced automatically from Git; Tier 3 is delivered only by merge request. Secrets: Tiers 1–2 in Git encrypted with SOPS + age, Tier 3 in Vault through External Secrets.

Figure 4 – Deployment tiers

© Terradue 2026 - The copyright in this document is vested in Terradue. This document may only be reproduced in whole or in part, or stored in a retrieval system, or transmitted in any form, or by any means electronic, mechanical, photocopying or otherwise, either with the prior permission of Terradue or in accordance with the terms of ESA Contract No. 4000144570/24/I-KE.

Table 6 – Deployment tiers

Tier	Contains	Installed by	Changed through
0	RKE2 cluster, node pools, Cilium, Longhorn, Rancher Monitoring	Rancher	Rancher UI / cluster configuration
1	Argo CD; cert-manager; Contour (external, internal); Valkey for GitLab; GitLab; Valkey for Harbor; Harbor; exposure of GitLab and Harbor (ClusterIssuer, Certificates, HTTPProxies)	Operator: helmfile (Argo CD), Argo CD Applications with **manual sync** (the others), kubectl (exposure) — all through bootstrap/bootstrap.sh	Merge request on bootstrap/, then the matching bootstrap.sh step (kubectl apply + diff + manual sync)
2	Crossplane core, provider-kubernetes, provider-helm (Terradue fork), DeploymentRuntimeConfig, ProviderConfigs	Operator: helmfile (never Argo CD)	Merge request on bootstrap/, helmfile diff / apply
3	Every other platform component (layers 10–70): Crossplane Releases and Objects in namespace platform	Argo CD (root Application → one Application per layer) + Crossplane	Merge request on platform/ — automated sync, prune and self-heal

RULE

Nothing manages its own manager. Argo CD, GitLab and Harbor are never Crossplane resources, and Crossplane is never an Argo CD Application: if Crossplane fails, Argo CD can still repair Tier 3 from Git; if Argo CD fails, it is reinstalled with helmfile; if GitLab is down, Argo CD keeps the last state and nothing is uninstalled.

Why Tier 1 Applications have no automated sync. GitLab may host the repository Argo CD reads, and Harbor hosts the packages Crossplane pulls: an automatic upgrade that broke them could not be rolled back through GitOps. They are changed only by an operator who reviews the diff and syncs.

Why Crossplane is not delivered by Argo CD. Removing a provider or its CRDs deletes every managed resource and uninstalls all of Tier 3. Keeping Crossplane in helmfile limits that blast radius to a deliberate operator action.

Table 7 – Where each component sits and why

Component	Tier	Reason
Argo CD	1	Delivers Tier 3; must not depend on what it delivers. helmfile install, port-forward only.
cert-manager, Contour	1	Harbor (cr.) and GitLab (git.) must be published with TLS before Crossplane can pull its packages from Harbor; Knative later reuses the same Contour.
Valkey (×2), GitLab, Harbor	1	Source of the configuration and of the artifacts of every other tier.
Crossplane + providers	2	Blast radius (see rule); controlled upgrades.
Knative Serving	3 (layer 30)	Not needed by Tiers 1–2; delivered by the Knative Operator chart.
Vault, ESO	3 (layer 10)	Needed only by Tier 3 secrets; Tier 1–2 secrets are SOPS files (section 4.4).

2.5 Component Inventory

Versions are those deployed on 28 October 2026. The status column uses: **Deployed** (running and verified), **Partial** (running, with temporary settings or pending checks), **To deploy** (manifests or inputs not yet complete). Section 12.1 gives the detail of what is missing.

© Terradue 2026 - The copyright in this document is vested in Terradue. This document may only be reproduced in whole or in part, or stored in a retrieval system, or transmitted in any form, or by any means electronic, mechanical, photocopying or otherwise, either with the prior permission of Terradue or in accordance with the terms of ESA Contract No. 4000144570/24/I-KE.

Table 8 – Component inventory

Component	Namespace	Version (chart / app)	Tier · layer	Status
RKE2, Cilium, CoreDNS	kube-system	v1.33.6+rke2r1 · Cilium 1.18.3	0	Provided
Longhorn	longhorn-system	1.10.2	0	Provided
Rancher Monitoring	cattle-monitoring-system	107.2.1+up69.8.2	0	Provided
Argo CD	argocd	chart 10.10.5 / v3.5.5	1	Deployed
cert-manager	cert-manager	v1.21.0	1	Deployed
Contour external + internal	contour-external, contour-internal	net-contour knative-v1.20.1 (Contour 1.33.1, Envoy 1.35.8)	1	Deployed
Valkey for GitLab	gitlab-system	valkey-helm @e840544 (Valkey 9.1.2)	1	Deployed
GitLab	gitlab-system	chart 9.11.13 / 18.11.12	1	Deployed
Valkey for Harbor	harbor-system	valkey-helm @e840544	1	Deployed
Harbor	harbor-system	chart 1.19.4 / 2.15.4	1	Deployed
Crossplane	crossplane-system	2.4.2	2	Deployed
provider-kubernetes	crossplane-system	v1.4.1 (community)	2	Deployed
provider-helm (Terradue)	crossplane-system	v1.0.0	2	Deployed
Vault	secret-management	chart 0.34.1 / 2.0.4, 3 pods Raft	3 · 10	Deployed
External Secrets Operator	eso	2.13.0	3 · 10	Deployed
Keycloak (keycloakx)	iam	7.4.0 / 26.8.1, 2 replicas	3 · 20	Deployed
OPA	opa	chart 0.2.5, image 1.21.2-envoy-3	3 · 20	Deployed (0.2.6 planned)
Knative Operator + Serving	knative-operator, knative-serving	chart 1.20.1 / Serving 1.20.1	3 · 30	Deployed
Strimzi operator	kafka-operator	1.1.0	3 · 40	Deployed
Kafka cluster	kafka	Kafka 4.2.1 (KRaft), 3 nodes	3 · 40	Deployed
Kafka UI	kafka-ui	chart 1.4.2	3 · 40	Deployed
STAC API (discover)	catalog	chart 2.0.7 / stac-fastapi-pgstac 6.2.1, pgSTAC 0.9.12	3 · 40	Deployed
TiTiler-pgSTAC	catalog	chart 3.0.6 / 2.1.0 (planned)	3 · 40	To deploy
Geobrowser	geobrowser	chart 1.0.0 / 2.1.7 (planned)	3 · 40	To deploy
STAC Browser, results catalogs	catalog	—	3 · 40	To deploy
Argo Workflows	processing	chart 0.45.26 / v3.7.2	3 · 50	Deployed
Argo Events	processing	chart 2.4.27	3 · 50	Deployed
Kafka event bus, CloudEvents webhook	processing	0.1.0 · 0.3.0	3 · 50	Deployed
Processing API (mate-api)	mate-api	chart 1.2.2, functions 1.2.0	3 · 50	Deployed
CWL runner templates	processing	—	3 · 50	To deploy
Portal, App Hub, Drive	portal, app-hub, drive	—	3 · 60	To deploy

© Terradue 2026 - The copyright in this document is vested in Terradue. This document may only be reproduced in whole or in part, or stored in a retrieval system, or transmitted in any form, or by any means electronic, mechanical, photocopying or otherwise, either with the prior permission of Terradue or in accordance with the terms of ESA Contract No. 4000144570/24/I-KE.

Component	Namespace	Version (chart / app)	Tier · layer	Status
Composition calrimate, application packages	crossplane-system, processing	—	3 · 70	To deploy
PostgreSQL (managed)	outside the cluster	16 + PostGIS 3.6.1, HA	provider	Provided
Object storage (managed)	outside the cluster	StorageGRID S3	provider	Provided

2.6 Key Design Decisions

Table 9 – Key design decisions

Decision	Rationale
Git is the only write path	Every change is reviewed, versioned and reversible; the commit history is the audit trail. Manual changes to Tier 3 are reverted automatically (self-heal).
Tiers with a single owner each	No circular dependency during bootstrap, upgrade or repair (section 2.4).
Tier 1 as Argo CD Applications with manual sync	Values reviewed in Git and diffed by Argo CD, but nothing changes GitLab, Harbor or the ingress without an operator.
Crossplane in helmfile, not in Argo CD	A wrong commit cannot remove the providers and with them every Tier 3 resource.
Crossplane v2 namespaced managed resources	Releases and Objects (*.m.* API groups) live in namespace platform next to their ProviderConfigs; one place to look.
Terradue fork of provider-helm	Drift detection on the objects rendered by a chart (enableDriftDetection), not only on chart version and values.
One Argo CD Application per layer	Layers start in order (sync-wave = layer number); a broken component blocks only its layer and the ones above; ApplyOutOfSyncOnly keeps syncs small.
Secrets: SOPS for Tiers 1–2, Vault for Tier 3	Tier 1–2 secrets are needed before Vault exists (also in disaster recovery); Tier 3 secrets never enter Git.
Harbor as the platform registry	Images, charts and Crossplane packages served from the platform itself; loaded by a scripted, reproducible seed (section 5.7).
No database and no bulk data in the cluster	Every relational database on the managed PostgreSQL 16 HA service, every file on StorageGRID. Bundled chart databases are always disabled.
Contour HTTPProxy for routing, OPA at Envoy	One HTTPProxy per host with TLS from cert-manager; authorisation decided by OPA before a request reaches a service.
Administrative UIs not published	Argo CD, Vault, Keycloak admin console, Kafka UI, Argo Workflows UI: kubectl port-forward only.
Kafka in KRaft mode	No ZooKeeper; three nodes act as controller and broker.
Platform components on the static pool	nodeSelector aig/node-pool=static on every platform component, so that processing load never evicts the control services.

© Terradue 2026 - The copyright in this document is vested in Terradue. This document may only be reproduced in whole or in part, or stored in a retrieval system, or transmitted in any form, or by any means electronic, mechanical, photocopying or otherwise, either with the prior permission of Terradue or in accordance with the terms of ESA Contract No. 4000144570/24/I-KE.

3 Infrastructure

3.1 Kubernetes Cluster

The cluster is an RKE2 cluster provisioned and managed by Rancher (management server svr-rke-prime-pan.ifxcloud.com). Rancher owns the cluster lifecycle: Kubernetes upgrades, node addition and removal, etcd snapshots and the system charts.

Table 10 – Cluster characteristics

Item	Value
Distribution	RKE2 v1.33.6+rke2r1
Container runtime	containerd 2.1.5
CNI	Cilium 1.18.3 (rke2-cilium)
Control plane	7 nodes with control-plane and etcd roles, tainted (no workloads)
Workers	26 nodes; the platform services run on the static pool nodo-worker-11 … 16 (section 3.2)
Operating system	Ubuntu 22.04 (workers, older control-plane nodes), Ubuntu 24.04 (newer control-plane nodes)
Kubeconfig	Personal kubeconfig downloaded from Rancher; context name copernicuslac-panama is assumed by the bootstrap tooling (override with export KUBE_CONTEXT=…)
Upgrades	Rancher plans through system-upgrade-controller

IMPORTANT

Version compatibility. Every chart and image of this issue was chosen for Kubernetes 1.33. Before a Kubernetes minor upgrade, check the removed APIs of every chart in bootstrap/ and platform/.

etcd and zones. The control plane has 7 etcd members of mixed size over two availability zones; the zone holding the majority is a single point of failure for the control plane.

3.2 Node Pools

Table 11 – Node pools

Pool	Nodes	Runs	Label / taint
Control plane	cp-cluster-k8s-copernicus-226 … 233 (7)	API server, etcd	RKE2 default, tainted
Static (platform)	nodo-worker-11 … 16 (6)	Argo CD, Crossplane, GitLab, Harbor, Valkey, Vault, ESO, Keycloak, OPA, Knative control plane, Kafka, STAC, processing control plane	**aig/node-pool=static** (applied); no taint
Processing	nodo-worker-17 … 29 (13)	Workflow and step pods	to be labelled and tainted [TBC-6]
Storage	nodo-worker-30 … 35 (6)	Longhorn replicas (10.4 TB data disk each)	longhorn-storage=true
Unassigned	nodo-worker-36 (1)	—	to be decided

The static label is set with:

```
for n in 11 12 13 14 15 16; do
```

© Terradue 2026 - The copyright in this document is vested in Terradue. This document may only be reproduced in whole or in part, or stored in a retrieval system, or transmitted in any form, or by any means electronic, mechanical, photocopying or otherwise, either with the prior permission of Terradue or in accordance with the terms of ESA Contract No. 4000144570/24/I-KE.

```
  kubectl label node nodo-worker-$n aig/node-pool=static --overwrite
done
```

- **Every platform component** declares `nodeSelector: {aig/node-pool: static}` in the chart key(s) the chart offers (global.nodeSelector, controller/webhook/cainjector, per-component keys). This is part of the authoring checklist (section 6.2).
- **DaemonSets** that must run everywhere (Envoy, Longhorn manager and CSI plugin, Cilium, monitoring exporters) have no node selector.
- **Labels must be applied on the Rancher machine pools** as well, so that they survive node replacement [TBC-6].

3.3 Storage

Longhorn 1.10.2, installed as a Rancher app, provides all in-cluster persistent storage.

Table 12 – StorageClasses used by the platform

StorageClass	Replicas	Access	Used by
longhorn (default)	3	RWO, RWX	Vault (data and audit), Keycloak themes (RWX), both Valkey instances, GitLab, Harbor; Portal / App Hub / Drive (planned)
longhorn-processing	1	RWO, RWX	**Kafka** (decision 2026-10-23: Kafka already replicates 3×, so one Longhorn replica avoids 9 copies); workflow working directories

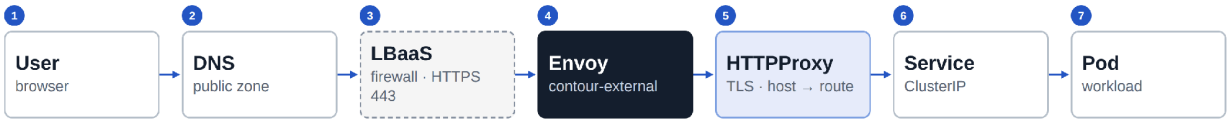
IMPORTANT

longhorn-processing has reclaimPolicy Delete: Kafka data is protected only by `deleteClaim: false` in the node pool. **Never delete the PVCs of namespace kafka by hand.**

Capacity: the storage pool provides 6 × 10.4 TB raw.

3.4 Network, DNS and Ingress

Public traffic follows the path in Figure 5: the provider's firewall and load balancer (outside the cluster) forward HTTP and HTTPS to the Envoy pods of contour-external, which terminate TLS and route by host name.



TLS terminates at Envoy with certificates issued by cert-manager (Let's Encrypt). Routing is declared per hostname in a Contour HTTPProxy. Cluster-local Knative routes use the second installation, contour-internal.

Figure 5 – Path of a user request

- **contour-external** serves all public host names; **contour-internal** serves the cluster-local routes created by Knative for the processing functions. Both are Tier 1 (section 5.3).
- Every HTTPProxy carries the annotation `projectcontour.io/ingress.class: contour-external`, sets `permitInsecure: false` and uses a Certificate from the ClusterIssuer letsencrypt-prod-contour (HTTP-01).
- The Envoy Service of type LoadBalancer stays `<pending>`: the load balancer is managed outside the cluster. Argo CD is configured to consider such a Service Healthy (section 5.2).
- Port 80 must be reachable from the Internet for the HTTP-01 challenge of every new certificate.

© Terradue 2026 - The copyright in this document is vested in Terradue. This document may only be reproduced in whole or in part, or stored in a retrieval system, or transmitted in any form, or by any means electronic, mechanical, photocopying or otherwise, either with the prior permission of Terradue or in accordance with the terms of ESA Contract No. 4000144570/24/I-KE.

RULE

DNS first. The public name of a service must resolve to the external load balancer **before** its step is run: the certificate cannot be issued otherwise. Do not work around a missing DNS record with CoreDNS entries or /etc/hosts: the kubelet of each node resolves registry names with the node DNS and needs a valid certificate.

Table 13 – DNS names of the platform

Name (.copernicuslac.aig.gob.pa)	Service	DNS
iam	Keycloak	Published
processing	Processing API	Published
discover	STAC API (catalog main)	Published
git	GitLab	**Published**
cr	Harbor	**Published**
view	TiTiler	Published
dashboards	Geobrowser	Published
download	download	**Pending**
results, www, apps, hub, drive	Results catalogs, Portal, App Hub, Drive	Published

3.5 Managed Data Services

Relational data and bulk files are never stored in the cluster. Two managed services, operated outside the cluster, hold them; the cluster reaches both over a private network.

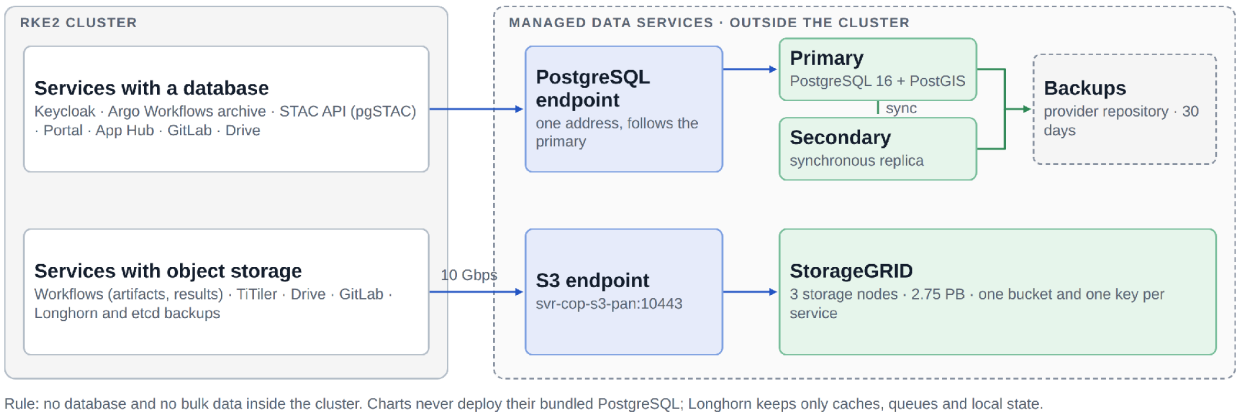


Figure 6 – Managed data services outside the cluster

3.5.1 PostgreSQL

Table 14 – Managed PostgreSQL service

Aspect	Value
Service	PostgreSQL 16 with PostGIS 3.6.1, btree_gist, unaccent (available in template1)
Topology	nodo01 **10.0.0.3** (primary) and nodo02 **10.0.0.4** (replica). Services connect to the primary address today.
Client network	Pods reach PostgreSQL from 192.168.80.0/24 (node addresses)

© Terradue 2026 - The copyright in this document is vested in Terradue. This document may only be reproduced in whole or in part, or stored in a retrieval system, or transmitted in any form, or by any means electronic, mechanical, photocopying or otherwise, either with the prior permission of Terradue or in accordance with the terms of ESA Contract No. 4000144570/24/I-KE.

Aspect	Value
Administration	pgAdmin with the DBA account (CREATEDB, CREATEROLE — not a superuser). The DBaaS team (contact: IFX) owns postgresql.conf, restarts and backups.
max_connections	**500 on both nodes (the replica must be ≥ the primary)**
shared_buffers / effective_cache_size	**8 GB on both nodes (about 13% of the 62 GB RAM, raised on 2026-10-26) / 16 GB on nodo01, 8 GB on nodo02 (to align, TBC-2)**

Table 15 – Databases on the managed service

Service	Database / role	Connection limit (role)	Credentials
GitLab	gitlabhq_production / gitlab (+ database postgres in pg_hba)	—	SOPS gitlab-postgres
Harbor	registry / harbor	—	SOPS harbor-postgres
Keycloak	keycloak / keycloak	30	Vault secrets/iam/keycloak-external-db
Argo Workflows archive	argowf / argowf	25	Vault secrets/processing/argo-postgres
STAC API (+ TiTiler)	pgstac / pgstac (pgSTAC 0.9.12)	50	Vault secrets/catalog/pgstac
Portal, App Hub, Drive, results catalogs	to create	—	—

Procedure for a new database (the same for every service):

1. Generate the password on the workstation and write it **first** to its destination (Vault for Tier 3, a SOPS file for Tiers 1–2), so that it is never typed twice: `P=$(openssl rand -base64 24 | tr -d '/+='), vault kv put secrets/<component>/<name> username=<role> password="$P", printf '%s' "$P" | pbcopy; unset P`.
2. In pgAdmin, on database postgres, with **nothing selected** in the editor (pgAdmin runs only the selection), one statement at a time (CREATE/DROP DATABASE cannot run inside the transaction pgAdmin opens for a block): `CREATE ROLE <role> LOGIN PASSWORD '<paste>' CONNECTION LIMIT <n>;` · `CREATE DATABASE <db> OWNER <role>;` · `REVOKE ALL ON DATABASE <db> FROM PUBLIC;`. Clear the password from the editor and the clipboard afterwards.
3. Add the line `host <db> <role> 192.168.80.0/24 scram-sha-256` to pg_hba.conf on **both** nodes (the file is not replicated) and reload: `sudo -u postgres psql -c 'SELECT pg_reload_conf();'`.
4. Check on each node: `SELECT line_number, error FROM pg_hba_file_rules WHERE '<db>' = ANY(database);` — the error column must be empty.

IMPORTANT

Connection budget. The sum of the maximum pools of every service (all replicas) must stay below max_connections. Check the usage with `SELECT count(*), current_setting('max_connections') FROM pg_stat_activity;`.

Check that the restart happened: `SELECT name, setting, pending_restart FROM pg_settings WHERE name IN ('max_connections','shared_buffers');` — pending_restart must be false on both nodes.

© Terradue 2026 - The copyright in this document is vested in Terradue. This document may only be reproduced in whole or in part, or stored in a retrieval system, or transmitted in any form, or by any means electronic, mechanical, photocopying or otherwise, either with the prior permission of Terradue or in accordance with the terms of ESA Contract No. 4000144570/24/I-KE.

3.5.2 Object Storage

Table 16 – Object storage service

Aspect	Value
Service	Private StorageGRID, S3 API, managed outside the cluster
Endpoint	https://svr-cop-s3-pan.ifxcloud.com:10443 — path-style bucket URLs (no virtual hosting)
Access model	one tenant user per service, in a group whose S3 policy is limited to that service's buckets; one access key per user, stored in SOPS (Tiers 1–2) or Vault (Tier 3)

Table 17 – Buckets

Bucket(s)	Used by	Policy file in the repository
copernicuslac-gitlab-{lfs, artifacts, uploads, packages, mr-diffs, ci-secure-files, dependency-proxy, registry, backups, tmp}	GitLab (versioning on only for -backups, 30-day expiry of old versions)	bootstrap/gitlab/storagegrid-policy.json
copernicuslac-harbor-registry	Harbor registry storage	bootstrap/harbor/storagegrid-policy.json
copernicuslac-argo-artifacts	Argo Workflows artifacts and logs (group/user argowf)	ops/storagegrid/argowf-group-policy.json
to create	TiTiler (read-only on the result buckets), Drive, results	—

To create a service user: Tenant Manager → Groups → create a local group with no tenant-management permission and the custom S3 group policy from the repository; Users → create the user in that group with "Deny access to Tenant Manager"; create an access key and write it directly to SOPS or Vault (never in chat or in a ticket).

IMPORTANT

Known issue of StorageGRID 12.0.0 to 12.0.0.4 (NetApp KB "StorageGRID returns incorrect 403 responses instead of 404 for HEAD/GET requests on non-existent objects"). A HEAD or GET on a key that does not exist returns 403 AccessDenied instead of 404, even when the user has s3:ListBucket on the bucket. Panama runs 12.0.0: the upgrade is requested to IFX (TBC-1).

Effect on Harbor: the registry relies on 404 to detect a missing object, so pushing an image that shares layers with an image already in another repository fails with HTTP 500 (registry log: s3aws: AccessDenied). Packages, charts and images with new layers are not affected. Check after the upgrade: s3cmd info s3://copernicuslac-harbor-registry/docker/does-not-exist must answer 404, then re-run ./bootstrap.sh harbor-seed.

3.6 Other External Dependencies

Table 18 – Other external dependencies

Dependency	Used by	If unavailable
CDSE	Input data for processing	New runs fail at stage-in
Let's Encrypt (ACME v2)	All public certificates (HTTP-01)	Renewals fail; certificates expire after 90 days
Public chart repositories and registries (argoproj.github.io, charts.jetstack.io, charts.gitlab.io, helm.goharbor.io, charts.crossplane.io, codecentric, knative.github.io, quay.io, ghcr.io, docker.io, xpkg.upbound.io, github.com)	Installs and upgrades	Running services unaffected; installs fail

© Terradue 2026 - The copyright in this document is vested in Terradue. This document may only be reproduced in whole or in part, or stored in a retrieval system, or transmitted in any form, or by any means electronic, mechanical, photocopying or otherwise, either with the prior permission of Terradue or in accordance with the terms of ESA Contract No. 4000144570/24/I-KE.

Dependency	Used by	If unavailable
cr.terradue.com (TEMPORARY)	Images of the processing API functions until they are copied to Harbor (section 12.3)	New pods of the processing API functions cannot pull their images

3.7 Access and Tooling

The Kubernetes API is reachable only through Rancher from authorised machines. Each operator uses a personal kubeconfig so that actions are attributable.

Table 19 – Operator tooling

Tool	Version	Use
kubectl	1.32–1.34 (version skew with the 1.33 server)	Inspection, bootstrap, recovery
helm + helm-diff plugin	≥ 3.14	Chart inspection, helmfile diff
helmfile	v1	Argo CD (Tier 1) and Crossplane (Tier 2)
argocd CLI	same minor as the server (3.5)	Diff and sync of Tier 1 Applications; `argocd --core` needs the kubeconfig context namespace set to argocd
sops + age	current	Tier 1–2 secrets (section 4.4)
vault CLI	2.x	Writing Tier 3 secrets through a port-forward
skopeo, oras	current	Harbor seed: images and Crossplane packages (skopeo), charts (oras)
yq, jq, curl, openssl	current	Scripts and checks
pgAdmin	current	Databases on the managed PostgreSQL

© Terradue 2026 - The copyright in this document is vested in Terradue. This document may only be reproduced in whole or in part, or stored in a retrieval system, or transmitted in any form, or by any means electronic, mechanical, photocopying or otherwise, either with the prior permission of Terradue or in accordance with the terms of ESA Contract No. 4000144570/24/I-KE.

4 Delivery Package and Repository

4.1 Principles

- **Git is the only write path.** The main branch of infra-copernicuslac-panama is the desired state. Tier 3 is changed only by merge request; Tiers 1–2 are changed by merge request and then applied by an operator with the documented step. No kubectl edit, helm upgrade or UI change on managed objects.
- **Everything is reproducible from the package.** The repository plus this guide rebuild the platform; the only inputs from outside are the secret values (age keys, Vault keys, passwords), the managed services and DNS.
- **Versions are pinned.** Chart versions, image tags, Git revisions and package tags are explicit; no latest, no ranges, no -develop tags.
- **Secrets never enter Git in clear.** Tiers 1–2: SOPS-encrypted files. Tier 3: Vault, referenced by ExternalSecrets.
- **Pull, not push.** Argo CD reads the repository from the in-cluster GitLab; nothing outside pushes into the cluster.

Table 20 – The two control loops of Tier 3

	Loop 1 — Argo CD	Loop 2 — Crossplane
Compares	Git ↔ Releases and Objects in namespace platform	Release / Object spec ↔ Helm release and objects
Triggered by	Repository poll (3 minutes)	Watch events and periodic poll (about 10 minutes for provider-helm)
On drift	selfHeal re-applies; prune deletes what left Git	Re-applies the manifest; runs helm upgrade
Reports	Sync status and Health of each layer Application	Conditions Synced and Ready

4.2 The Delivery Package

The operations team receives a zip of the repository infra-copernicuslac-panama. It contains everything needed to rebuild the platform, **except** the items in the second table, which are delivered separately.

Table 21 – Content of the delivery package

Item	Content
bootstrap/	Tiers 1–2: bootstrap.sh, helmfile.yaml.gotmpl, values and Argo CD Applications of every Tier 1 component, exposure manifests, Crossplane charts, SOPS-encrypted secrets and their templates, SQL and S3 policies of GitLab and Harbor, harbor/seed.yaml
apps/	One Argo CD Application per Tier 3 layer
platform/	Tier 3 Releases and Objects, one folder per layer and one sub-folder per component, each with a README
ops/	Operational scripts and references: Vault init/unseal/configure, Keycloak realm export, StorageGRID policies, security tests
docs/	Deployment notes (decisions and issues met), source of this issue
.sops.yaml	SOPS rules: which files are encrypted (path_regex \.sops\.yaml$) and which fields (encrypted_regex ^(data\|stringData)$), public age keys

© Terradue 2026 - The copyright in this document is vested in Terradue. This document may only be reproduced in whole or in part, or stored in a retrieval system, or transmitted in any form, or by any means electronic, mechanical, photocopying or otherwise, either with the prior permission of Terradue or in accordance with the terms of ESA Contract No. 4000144570/24/I-KE.

Table 22 – Delivered separately (never in the zip, never in Git)

Item	How it is delivered
Age private keys (.sops/age.key, keys.txt) and the team disaster-recovery key	Team password manager, out of band
Vault unseal keys (5) and root token (ops/vault/vault-init.sops.yaml, git-ignored)	Team password manager; the file is deleted from the workstation after storing
.envrc (local environment)	Each operator creates their own
bootstrap/harbor/archives/ (git-ignored local copies of packages and charts for the Harbor seed)	Recreated with the commands documented in harbor/seed.yaml, or handed over as a separate archive
Passwords of the managed services (DBA account, StorageGRID tenant)	Provider / team password manager

IMPORTANT
Before creating the zip, check that no secret is in clear: `grep -L 'ENC\[' bootstrap/secrets/*.sops.yaml` must print nothing, and `git status --ignored` must show .sops/, .envrc and harbor/archives/ as ignored.

4.3 Repository Structure

```
infra-copernicuslac-panama/
├── README.md                    # tiers, entry point
├── .sops.yaml                   # SOPS rules + public age keys
├── bootstrap/                   # Tiers 1-2 — applied by the operator, never read by Argo CD
│   ├── bootstrap.sh             # every step, idempotent (section 5.1)
│   ├── helmfile.yaml.gotmpl     # Argo CD (tier=1), Crossplane + packages + configs (tier=2)
│   ├── argocd/values.yaml       # health checks for Crossplane, Service override,
nodeSelector
│   ├── cert-manager/application.yaml
│   ├── contour/application.yaml # net-contour config/contour @ knative-v1.20.1
│   ├── valkey/application.yaml  # Valkey for GitLab (valkey-helm @ commit)
│   ├── gitlab/                  # application.yaml, database.sql, storagegrid-policy.json
│   ├── harbor/                  # application, valkey, database.sql, storagegrid-policy,
│                                # seed.yaml, archives/ (git-ignored)
│   ├── exposure/                # cluster-issuer, gitlab, harbor (Certificate + HTTPProxy)
│   ├── crossplane/values.yaml
│   ├── crossplane-packages/     # local chart: Providers, DeploymentRuntimeConfig, RBAC
│   ├── crossplane-providerconfigs/
│   ├── root-app.yaml            # Tier 3 entry point
│   └── secrets/                 # *.sops.yaml, templates/, gen-harbor-secrets.sh, README
├── apps/                        # 10-core … 70-application-packages (one Application each)
├── platform/                    # Tier 3 — Releases and Objects, namespace platform
│   ├── _template/               # conventions + example component (never synced)
│   ├── 10-core/                 # vault/, external-secrets/
│   ├── 20-identity/             # keycloak/, opa/
│   ├── 30-serving/              # knative-operator/, knative-serving/
│   ├── 40-data/                 # kafka/, kafka-ui/, stac-api/, titiler-pgstac/,
stac-browser/
│   ├── 50-processing/           # namespaces, argo-workflows, argo-events, event-bus,
mate-api
│   ├── 60-user-services/
│   └── 70-application-packages/
├── ops/
│   ├── vault/                   # init.sh, unseal.sh, configure.sh, lib.sh, README.md
│   ├── keycloak/                # CopernicusLAC-Platform-realm.json (partial export)
│   ├── storagegrid/             # argowf-group-policy.json
│   └── tests/                   # opa-negative-tests.sh
└── docs/deployment-notes.md
```

- One resource per file in platform/; the file name is the resource name.
- bootstrap/ is never read by Argo CD from Git: the root Application points at apps/ only.

© Terradue 2026 - The copyright in this document is vested in Terradue. This document may only be reproduced in whole or in part, or stored in a retrieval system, or transmitted in any form, or by any means electronic, mechanical, photocopying or otherwise, either with the prior permission of Terradue or in accordance with the terms of ESA Contract No. 4000144570/24/I-KE.

- Removing a file from platform/ removes the resource (prune), except resources annotated Prune=false (data holders: Vault, Kafka, KnativeServing, Argo releases).
- Files ending in .disabled are kept in Git but not synced (Argo CD reads only .yaml/.yml/.json): used for prepared but inactive configuration (Argo Workflows SSO).
- Every component folder has a README with its prerequisites (database, Vault paths, DNS) and checks.

4.4 Secrets Model

Table 23 – Where secrets live

Secrets of	Live in	Applied by
Tiers 1–2 (Argo CD, Valkey, GitLab, Harbor, Crossplane)	bootstrap/secrets/*.sops.yaml, encrypted with SOPS + age	./bootstrap.sh secrets (sops -d | kubectl apply)
Tier 3 (everything in platform/)	Vault KV v2, mount secrets/	External Secrets Operator, through the ClusterSecretStore vault-backend; only namespaces labelled eso=enabled

Tier 1–2 secrets cannot come from Vault: Vault is a Tier 3 component and does not exist when they are needed, also during disaster recovery. Each operator has an age key; the team also has a disaster-recovery key whose private part lives only in the password manager. Every file is encrypted for all public keys listed in .sops.yaml.

```
# one-time, per operator (macOS path; Linux: ~/.config/sops/age/keys.txt)
age-keygen -y "$HOME/Library/Application Support/sops/age/keys.txt"  # → add to .sops.yaml
sops updatekeys bootstrap/secrets/*.sops.yaml        # after a .sops.yaml change

# create a secret from its template
cp bootstrap/secrets/templates/gitlab-postgres.yaml
bootstrap/secrets/gitlab-postgres.sops.yaml
$EDITOR bootstrap/secrets/gitlab-postgres.sops.yaml      # replace every CHANGE-ME
sops --encrypt --in-place bootstrap/secrets/gitlab-postgres.sops.yaml

# change one field of an encrypted file (editing the encrypted text by hand breaks the MAC)
sops set bootstrap/secrets/helm-repo-harbor.sops.yaml \
  '["metadata"]["namespace"]' '"platform"'

# check before every commit: must print nothing
grep -L 'ENC\[' bootstrap/secrets/*.sops.yaml
```

Table 24 – Tier 1–2 secrets inventory

File	Namespace / Secret	Content
gitlab-postgres	gitlab-system / gitlab-postgres	password of role gitlab
gitlab-initial-root-password	gitlab-system	first root login
gitlab-object-storage, -plain, gitlab-registry-storage	gitlab-system	StorageGRID key of user gitlab (Rails, backups, registry)
gitlab-valkey-users	gitlab-system	Valkey ACL passwords (default, replicator) and Sentinel password
gitlab-keycloak	gitlab-system	OIDC provider of GitLab (client gitlab, realm CopernicusLAC-Platform)
gitlab-rails-secret	gitlab-system / copernicuslac-gitlab-rails-secret	generated by GitLab, exported after the first install (section 5.4) — without it a database restore is useless
argocd-repo-infra	argocd	GitLab deploy token (read_repository) of this repository

© Terradue 2026 - The copyright in this document is vested in Terradue. This document may only be reproduced in whole or in part, or stored in a retrieval system, or transmitted in any form, or by any means electronic, mechanical, photocopying or otherwise, either with the prior permission of Terradue or in accordance with the terms of ESA Contract No. 4000144570/24/I-KE.

File	Namespace / Secret	Content
harbor-internal, harbor-token-service, harbor-valkey-sentinel	harbor-system	generated by secrets/gen-harbor-secrets.sh (admin password, internal secrets, PKCS#1 token-service key)
harbor-postgres, harbor-registry-storage	harbor-system	role harbor password, StorageGRID key of user harbor
harbor-crossplane-pull	crossplane-system	robot$crossplane+pull (project robot, pull only, project crossplane) — written by ./bootstrap.sh harbor-robots
helm-repo-harbor	platform	**robot$helm-pull (pull only, chart projects opa-authz, strimzi-kafka, mate, seda, stac-utils): Releases' pullSecretRef — written by ./bootstrap.sh harbor-robots**
harbor-oidc	harbor-system	**client secret of Keycloak client harbor, read by ./bootstrap.sh harbor-oidc**

RULE
Never commit a file of templates/ with real values or a *.sops.yaml that is not encrypted. Never put Tier 3 secrets in bootstrap/secrets/. Never paste a secret in a chat, a ticket or a command line: a value exposed this way is rotated immediately.

4.5 End-to-end Sequence

A cluster is brought from empty to fully delivered in the order below; the same order is used for disaster recovery (section 11.5). Each step depends only on the previous ones and is idempotent.

Table 25 – End-to-end deployment sequence

#	Step	How	Section	Done when
1	Cluster, node pools, static label, Longhorn, Monitoring	Rancher; kubectl label	3.2	Nodes Ready, label set
2	External prerequisites: databases, pg_hba, buckets, users, DNS of git. and cr.	pgAdmin, StorageGRID, DNS	3.5	Checks of section 3.5
3	Namespaces and SOPS secrets	./bootstrap.sh namespaces · secrets	5.1	Secrets present
4	Argo CD	./bootstrap.sh argocd	5.2	argocd-server Running
5	cert-manager, Contour	./bootstrap.sh cert-manager · contour	5.3	Applications Synced/Healthy
6	Valkey and GitLab	./bootstrap.sh valkey · gitlab	5.4	GitLab login; rails secret exported
7	Repository on GitLab; deploy token	git push; SOPS argocd-repo-infra	5.4	Repository readable from the cluster
8	Valkey for Harbor and Harbor	./bootstrap.sh harbor-valkey · harbor	5.5	Harbor API answers
9	Exposure of GitLab and Harbor	./bootstrap.sh exposure	5.6	Certificates Ready, HTTPS answers

© Terradue 2026 - The copyright in this document is vested in Terradue. This document may only be reproduced in whole or in part, or stored in a retrieval system, or transmitted in any form, or by any means electronic, mechanical, photocopying or otherwise, either with the prior permission of Terradue or in accordance with the terms of ESA Contract No. 4000144570/24/I-KE.

#	Step	How	Section	Done when
10	Harbor seed and robot accounts	./bootstrap.sh harbor-seed · harbor-robots	5.7	Artifacts in Harbor, xpkg annotations present; robot secrets in SOPS
11	Crossplane (Tier 2)	./bootstrap.sh crossplane	5.8	Providers Installed/Healthy
12	Tier 3 root Application	./bootstrap.sh root-app	6.1	Layer Applications created
13	Layer 10: Vault init, unseal, configure	ops/vault/*.sh	7.1	vault-backend Ready
14	Layers 20–70, each with its prerequisites	Vault secrets, databases, Keycloak clients, then Git; after layer 20: ./bootstrap.sh harbor-oidc	7–10	Each layer Synced/Healthy

TODAY

In Panama every step up to layer 50 is done (Tier 1 published on 26 October 2026). The images of the processing API functions are still pulled from cr.terradue.com: copying them to Harbor is blocked by the StorageGRID issue of section 3.5.2 (TBC-1).

© Terradue 2026 - The copyright in this document is vested in Terradue. This document may only be reproduced in whole or in part, or stored in a retrieval system, or transmitted in any form, or by any means electronic, mechanical, photocopying or otherwise, either with the prior permission of Terradue or in accordance with the terms of ESA Contract No. 4000144570/24/I-KE.

5 Tiers 1 and 2 — Bootstrap

Everything in this chapter is applied by the operator from bootstrap/, never synchronised automatically from Git. Prerequisites on the workstation: the tools of section 3.7, the kubeconfig context copernicuslac-panama, an age key able to decrypt bootstrap/secrets/, and network access from the cluster to the public chart repositories of section 4.

5.1 bootstrap.sh

bootstrap/bootstrap.sh wraps every Tier 1–2 operation. Each step checks its prerequisites (secrets, health of the previous step), is idempotent and can be re-run. Tier 1 Applications have no automated sync: the script starts the sync itself and waits until the Application is Synced and Healthy.

Table 26 – Steps of bootstrap.sh

Step	What it does	Requires
namespaces	Creates argocd, gitlab-system, harbor-system, crossplane-system, platform	—
secrets	Decrypts and applies every secrets/*.sops.yaml	sops, age key
argocd	helmfile -l name=argocd apply; waits for argocd-server	secrets
cert-manager	Applies the Application, syncs, waits Healthy (adopts an existing install with the same names)	argocd
contour	Same for contour-external and contour-internal	argocd
valkey	Valkey for GitLab; waits Healthy	secret gitlab-valkey-users
gitlab	GitLab Application and first sync (15–25 min)	Valkey Healthy, GitLab secrets
harbor-valkey	Valkey for Harbor; waits Healthy	secret harbor-valkey-sentinel
harbor	Harbor Application and first sync (5–10 min)	Harbor Valkey Healthy, Harbor secrets
exposure	ClusterIssuer, Certificates and HTTPProxies of git. and cr.; waits for the certificates	cert-manager, Contour Healthy; DNS
harbor-seed	Copies the artifacts of harbor/seed.yaml into Harbor	Harbor; skopeo, oras, yq, curl
harbor-robots	Creates the read-only robot accounts robot$crossplane+pull and robot$helm-pull and writes their secrets straight into the SOPS files (shown only once); existing robots are kept, ROTATE=1 regenerates	Harbor published; sops, yq, curl
harbor-oidc	Harbor sign-in through Keycloak: writes the OIDC configuration (client harbor, groups claim, admin group admin) through the Harbor API; the local admin stays as break-glass account	Harbor published; Keycloak (layer 20); secrets/harbor-oidc.sops.yaml
crossplane	helmfile -l tier=2 apply (Crossplane, packages, ProviderConfigs)	pull secrets of the packages
root-app	Applies bootstrap/root-app.yaml (Tier 3)	providers Healthy, secret argocd-repo-infra
status	Prints the state of every Tier 1–2 component	—
all	namespaces → secrets → argocd → cert-manager → contour → valkey → gitlab → harbor-valkey → harbor	—

```
cd bootstrap
./bootstrap.sh namespaces && ./bootstrap.sh secrets
./bootstrap.sh argocd
./bootstrap.sh cert-manager && ./bootstrap.sh contour
./bootstrap.sh valkey && ./bootstrap.sh gitlab
```

© Terradue 2026 - The copyright in this document is vested in Terradue. This document may only be reproduced in whole or in part, or stored in a retrieval system, or transmitted in any form, or by any means electronic, mechanical, photocopying or otherwise, either with the prior permission of Terradue or in accordance with the terms of ESA Contract No. 4000144570/24/I-KE.

```
./bootstrap.sh harbor-valkey && ./bootstrap.sh harbor
./bootstrap.sh exposure          # needs DNS of git. and cr.
./bootstrap.sh harbor-seed
./bootstrap.sh harbor-robots # commit the changed secrets/*.sops.yaml
./bootstrap.sh crossplane
./bootstrap.sh root-app
./bootstrap.sh harbor-oidc # once Keycloak (layer 20) is up
./bootstrap.sh status
```

5.2 Argo CD

Table 27 – Argo CD

Aspect	Value
Release	argocd — chart argo/argo-cd 10.10.5 (Argo CD v3.5.5), helmfile (bootstrap/helmfile.yaml.gotmpl, label tier=1)
Namespace	argocd, all components on the static pool (global.nodeSelector)
Access	**Not exposed.** `kubectl -n argocd port-forward svc/argocd-server 8080:80`; initial password in Secret argocd-initial-admin-secret (rotate it, then delete the Secret)
Repository	http://copernicuslac-gitlab-webservice-default.gitlab-system.svc:8181/copernicuslac/infra-copernicuslac-panama.git (in-cluster GitLab), credentials Secret argocd-repo-infra (deploy token, SOPS)
Changes	helmfile -l name=argocd diff, then apply

bootstrap/argocd/values.yaml adds the health checks that make Argo CD understand the platform:

Table 28 – Argo CD customisations

Setting	Why
Health of Release, Object, ApplicationPackage (Lua)	Healthy only when Crossplane reports Synced=True and Ready=True; Degraded when Synced=False. Without it a Release is Healthy as soon as it exists, even if Helm failed.
Health of argoproj.io/Application (Lua)	The root Application waits for each layer Application to be Healthy before the next sync wave: layers start in order.
Health of Service	The external load balancer is outside the cluster: Services of type LoadBalancer stay <pending> and would keep Contour Progressing forever. A Service is Healthy once it exists.
global.nodeSelector aig/node-pool: static	Platform placement (section 3.2).

NOTE
The argocd CLI in core mode (`argocd --core app get …`) talks to the API server directly and needs the current kubeconfig context namespace to be argocd: `kubectl config set-context --current --namespace argocd`.

5.3 cert-manager and Contour

Table 29 – cert-manager and Contour

Aspect	cert-manager	Contour
Application	bootstrap/cert-manager/application.yaml	bootstrap/contour/application.yaml

© Terradue 2026 - The copyright in this document is vested in Terradue. This document may only be reproduced in whole or in part, or stored in a retrieval system, or transmitted in any form, or by any means electronic, mechanical, photocopying or otherwise, either with the prior permission of Terradue or in accordance with the terms of ESA Contract No. 4000144570/24/I-KE.

Aspect	cert-manager	Contour
Source	chart jetstack/cert-manager v1.21.0, values inline	github.com/knative-extensions/net-contour, path config/contour, targetRevision knative-v1.20.1 (Contour 1.33.1, Envoy 1.35.8), files external.yaml and internal.yaml
Settings	CRDs installed and kept; startupapicheck off; controller, webhook, cainjector on the static pool	ServerSideApply; ignoreDifferences on CRD /spec/preserveUnknownFields; two installations contour-external and contour-internal; Envoy DaemonSet on every worker
Upgrade	Change targetRevision, diff, sync; read the release notes	Together with Knative Serving (same minor): targetRevision here, then the Knative Operator and KnativeServing (section 7.7)

NOTE
Argo CD reports a RepeatedResourceWarning on the Contour Application: the CRDs and one ClusterRole are present in both external.yaml and internal.yaml. It is expected and harmless.

5.4 Valkey and GitLab

Table 30 – Valkey for GitLab

Aspect	Value
Source	github.com/valkey-io/valkey-helm at commit e840544 (Valkey 9.1.2): 3 Valkey nodes, 3 Sentinels, 3 HAProxy
Security	ACL users default (GitLab) and replicator, Sentinel password — Secret gitlab-valkey-users (SOPS)
Availability	PodDisruptionBudget; preStop asks Sentinel for a coordinated failover before a master stops
Check	`valkey-cli -h gitlab-valkey-haproxy --user default --pass "$PW" info replication` → role:master, connected_slaves:2

Table 31 – GitLab

Aspect	Value
Source	chart gitlab/gitlab 9.11.13 (GitLab 18.11.12), Application from the public Helm repository, values inline
Why not 19.x	GitLab 19 (chart 10.x) requires PostgreSQL 17; the managed service is PostgreSQL 16. Path: PostgreSQL 17 by the provider → upgrade 18.11 → 19.x following the GitLab upgrade path, after a backup.
Database	gitlabhq_production on the managed PostgreSQL (bootstrap/gitlab/database.sql); pg_hba must also allow the database postgres (GitLab checks)
Object storage	StorageGRID buckets copernicuslac-gitlab-* (section 3.5.2)
Redis	Valkey above (no bundled Redis)
Identity	Keycloak OIDC (Secret gitlab-keycloak, client gitlab, realm CopernicusLAC-Platform)
Host names	git.copernicuslac.aig.gob.pa (published), registry.copernicuslac.aig.gob.pa (registry enabled, not exposed: no Certificate/HTTPProxy yet)
Known diff	ignoreDifferences on the Gitaly StatefulSet volumeClaimTemplates (apiVersion, kind, status), otherwise OutOfSync forever

© Terradue 2026 - The copyright in this document is vested in Terradue. This document may only be reproduced in whole or in part, or stored in a retrieval system, or transmitted in any form, or by any means electronic, mechanical, photocopying or otherwise, either with the prior permission of Terradue or in accordance with the terms of ESA Contract No. 4000144570/24/I-KE.

After the first install:

1. **Export the Rails secret** (it encrypts CI variables, 2FA seeds and tokens; without it a database restore is useless): `kubectl -n gitlab-system get secret copernicuslac-gitlab-rails-secret -o yaml | yq 'del(.metadata.uid, .metadata.resourceVersion, .metadata.creationTimestamp, .metadata.managedFields, .metadata.annotations, .metadata.labels, .metadata.ownerReferences)' > secrets/gitlab-rails-secret.sops.yaml`, then `sops --encrypt --in-place secrets/gitlab-rails-secret.sops.yaml` and commit.
2. Log in through `kubectl -n gitlab-system port-forward svc/copernicuslac-gitlab-webservice-default 8181:8181` (http://localhost:8181, user root, initial password from SOPS). Until git. is published the browser login can fail because of Secure cookies on plain HTTP: use the API or HTTPS.
3. Create the group copernicuslac and the private project infra-copernicuslac-panama, push the repository, create a **deploy token** with read_repository and store it as secrets/argocd-repo-infra.sops.yaml.
4. After ./bootstrap.sh exposure, sign in with the CopernicusLAC button (Keycloak). Users are created in Keycloak with a short username, not the e-mail: GitLab links the account on preferred_username. The first sign-in creates the GitLab user blocked (blockAutoCreatedUsers): approve it in Admin → Users → Pending approval. A second sign-in before the approval fails with 422 "Email has already been taken".
5. Git access for operators: member of the group copernicuslac (Developer to open merge requests, Maintainer to merge into the protected main) and a personal access token (scope write_repository) used as Git password over HTTPS, remote https://git.copernicuslac.aig.gob.pa/copernicuslac/infra-copernicuslac-panama.git. SSH is not exposed; the port-forward of step 2 is only for the bootstrap and disaster recovery.

Changing GitLab: merge request on gitlab/application.yaml; toolbox backup (`kubectl -n gitlab-system exec deploy/copernicuslac-gitlab-toolbox -- backup-utility`) before an upgrade; `kubectl apply -f gitlab/application.yaml`; `argocd app diff gitlab`; `argocd app sync gitlab`; if needed `argocd app rollback gitlab` (works with GitLab down).

5.5 Harbor

Table 32 – Harbor

Aspect	Value
Source	chart harbor/harbor 1.19.4 (Harbor 2.15.4), Application from helm.goharbor.io, values inline
External URL	https://cr.copernicuslac.aig.gob.pa (registry. belongs to GitLab)
Database	registry on the managed PostgreSQL (bootstrap/harbor/database.sql); pool maxIdle 10 / maxOpen 50
Storage	StorageGRID bucket copernicuslac-harbor-registry, path-style
Redis	Dedicated Valkey harbor-valkey (bootstrap/harbor/valkey.yaml), **without data password** — see note
Secrets	All generated values as existing Secrets: harbor-internal, harbor-token-service (PKCS#1 key), harbor-postgres, harbor-registry-storage (secrets/gen-harbor-secrets.sh + templates)
Identity	Keycloak OIDC (client harbor, confidential, redirect https://cr.copernicuslac.aig.gob.pa/c/oidc/callback, client scope groups with full path off, admin group admin, user claim preferred_username, auto-onboard), stored in the Harbor database and written by ./bootstrap.sh harbor-oidc; local admin kept as break-glass account
Robot accounts	robot$crossplane+pull (project robot, project crossplane: listed under Projects → crossplane → Robot Accounts, not under Administration) and robot$helm-pull (system robot, chart projects), both pull only, created by ./bootstrap.sh harbor-robots
Placement	Every component on the static pool

© Terradue 2026 - The copyright in this document is vested in Terradue. This document may only be reproduced in whole or in part, or stored in a retrieval system, or transmitted in any form, or by any means electronic, mechanical, photocopying or otherwise, either with the prior permission of Terradue or in accordance with the terms of ESA Contract No. 4000144570/24/I-KE.

NOTE

Why every Harbor secret is an existing Secret and Valkey has no password. Argo CD renders charts without Helm lookup: values generated by the chart would change at every render, and the chart writes the Redis password into ConfigMaps. The Valkey data endpoint is therefore reachable without password from pods of the cluster (no NetworkPolicy, decision of the platform team).

Token-service key in PKCS#1. OpenSSL 3 generates PKCS#8 by default and Harbor answers 500 on every token request; gen-harbor-secrets.sh uses `openssl rsa -traditional`.

DONE WHEN

`argocd app get harbor-valkey` and `argocd app get harbor` → Synced / Healthy; the API answers on `kubectl -n harbor-system port-forward svc/harbor 8282:80` with the admin password from SOPS. Image pushes are validated only through HTTPS, after the exposure step.

5.6 Exposure of Tier 1 Services

bootstrap/exposure/ holds the ClusterIssuer letsencrypt-prod-contour (Let's Encrypt, HTTP-01 through contour-external, account key Secret issuer-account-key-copernicuslac) and, for git. and cr., a Certificate (ECDSA P-256, key rotated at every renewal) and an HTTPProxy. Argo CD is deliberately not in this folder.

```
apiVersion: projectcontour.io/v1
kind: HTTPProxy
metadata:
  name: harbor
  namespace: harbor-system
  annotations:
    projectcontour.io/ingress.class: contour-external
spec:
  virtualhost:
    fqdn: cr.copernicuslac.aig.gob.pa
    tls: {secretName: harbor-tls}
  routes:
    - conditions: [{prefix: /}]
      permitInsecure: false
      services: [{name: harbor, port: 80}]
      timeoutPolicy: {response: infinity, idle: 5m}     # large image layers
```

Run `./bootstrap.sh exposure` once the DNS records exist; the step waits for the certificates. Check with `kubectl get certificate -A` (READY True) and `curl -sI https://cr.copernicuslac.aig.gob.pa`.

5.7 Harbor Seed

Harbor must hold the Crossplane packages, charts and images that the platform pulls. bootstrap/harbor/seed.yaml lists them; `./bootstrap.sh harbor-seed` creates the missing private projects and copies every missing artifact (existing ones are skipped).

Table 33 – Content of harbor/seed.yaml

Kind	Harbor project	Artifacts
Crossplane packages	crossplane	provider-helm-terradue:v1.0.0 (local copy dir:), provider-kubernetes:v1.4.1 (xpkg.upbound.io)
Charts (OCI)	opa-authz	opa 0.2.5
	strimzi-kafka	kafka-ui 1.4.2
	mate	mate-api 1.2.2

© Terradue 2026 - The copyright in this document is vested in Terradue. This document may only be reproduced in whole or in part, or stored in a retrieval system, or transmitted in any form, or by any means electronic, mechanical, photocopying or otherwise, either with the prior permission of Terradue or in accordance with the terms of ESA Contract No. 4000144570/24/I-KE.

Kind	Harbor project	Artifacts
	seda	kafka-event-bus 0.1.0, cloud-event-webhook 0.3.0
	stac-utils	stac-fastapi-pgstac 2.0.7
To add	mate, stac-utils, glowstac	mate function images 1.2.0 (listed; copy blocked by the StorageGRID issue, TBC-1), titiler-pgstac, geobrowser

RULE
Copy Crossplane packages only with `skopeo copy --preserve-digests` from `docker://…` or from a local `dir:` copy. `docker save/load/push`, `oci-archive:` and `docker-archive:` drop the io.crossplane.xpkg layer annotations and the package can no longer be installed; seed entries with xpkg: true are checked after the copy. Charts are pushed with oras using the Helm OCI media types.

If cr. is not reachable (DNS or certificate not ready, disaster recovery), the step switches Harbor's externalURL to http://localhost:8282, copies through a port-forward and **always** restores externalURL, also on error or Ctrl-C. To add an artifact: merge request on seed.yaml (and the local file in harbor/archives/ when needed), then re-run the step.

5.8 Crossplane (Tier 2)

Table 34 – Tier 2 releases (helmfile, label tier=2)

Release	Content	Version
crossplane	Crossplane core, 2 replicas, static pool (crossplane/values.yaml)	chart crossplane-stable/crossplane 2.4.2
crossplane-packages	Providers, DeploymentRuntimeConfig (nodeSelector static on container package-runtime), cluster-admin binding of the provider ServiceAccounts	provider-kubernetes v1.4.1, provider-helm (Terradue) v1.0.0
crossplane-providerconfigs	ProviderConfigs kubernetes-provider and helm-provider in namespace platform (presync hook waits for the providers to be Healthy)	—

```
./bootstrap.sh crossplane                          # = helmfile -l tier=2 apply
kubectl get providers.pkg.crossplane.io              # INSTALLED True, HEALTHY True
kubectl -n platform get providerconfigs.helm.m.terradue.crossplane.io \
  providerconfigs.kubernetes.m.crossplane.io
```

NOTE
Both providers are pulled from Harbor: cr.copernicuslac.aig.gob.pa/crossplane/…, pull secret harbor-crossplane-pull (robot$crossplane+pull). For a disaster recovery before Harbor is back, crossplane-packages/values.yaml lists the upstream sources in a comment: xpkg.upbound.io (public) and cr.terradue.com (read-only robot of cr.terradue.com, Secret from secrets/templates/cr-terradue-pull.yaml). Changing the source changes only where the package is pulled from, not the providers.

Upgrading Crossplane or a provider: one component at a time, `helmfile -l name=<release> diff`, apply, and wait for the new ProviderRevision to be Healthy before the next. Never uninstall a provider: its CRDs and every managed resource would be deleted.

© Terradue 2026 - The copyright in this document is vested in Terradue. This document may only be reproduced in whole or in part, or stored in a retrieval system, or transmitted in any form, or by any means electronic, mechanical, photocopying or otherwise, either with the prior permission of Terradue or in accordance with the terms of ESA Contract No. 4000144570/24/I-KE.

6 Tier 3 — Platform Delivery

6.1 Root Application and Layers

bootstrap/root-app.yaml (applied once with `./bootstrap.sh root-app`) reads apps/ and creates one Application per layer. Each layer Application syncs every file under platform/<layer>/ into namespace platform.

Table 35 – Layer Applications

Application	Wave	Content
10-core	10	Vault, External Secrets Operator, ClusterSecretStore
20-identity	20	Keycloak, OPA
30-serving	30	Knative Operator, KnativeServing
40-data	40	Strimzi + Kafka, Kafka UI, STAC API, TiTiler-pgSTAC, Geobrowser, STAC Browser
50-processing	50	Argo Workflows, Argo Events, event bus, CloudEvents webhook, processing API, CWL runner
60-user-services	60	Portal, App Hub, Drive
70-application-packages	70	Composition calrimate, ApplicationPackages

- Sync policy of every layer: automated, prune, selfHeal; options CreateNamespace=false, ApplyOutOfSyncOnly (a commit touching one component re-applies only that component), ServerSideApply, SkipDryRunOnMissingResource (CRDs installed by a Release of the same layer); retry 5 times with backoff 30 s → 5 min.
- **No resources-finalizer** on the root and layer Applications: deleting an Application by mistake never uninstalls its layer.
- The root Application waits for each layer to be Healthy (Application health of section 5.2) before starting the next wave; a failing component blocks only its layer and the ones above.
- Inside a layer, the annotation argocd.argoproj.io/sync-wave orders the components (−1 namespaces, 0 secrets and prerequisites, 1 Releases, 2 Objects that use the CRDs or Services of the Releases).

6.2 Authoring Conventions

platform/_template/ contains the rules and an example component (never synced). Copy example/ to platform/<layer>/<component>/, rename and adapt.

Table 36 – Authoring checklist

Rule	Detail
Kinds	Charts: Release (helm.m.terradue.crossplane.io/v1beta1, ProviderConfig helm-provider). Anything else (Namespace, Certificate, HTTPProxy, ExternalSecret, CR of an operator): Object (kubernetes.m.crossplane.io/v1alpha1, ProviderConfig kubernetes-provider). Every resource in namespace platform; one resource per file.
Namespace	An Object with the Namespace, sync-wave −1, managementPolicies [Observe, Create, Update, LateInitialize] (never deleted). Label **eso: enabled** if the namespace holds an ExternalSecret.
Secrets	Never in values. An ExternalSecret (Object) reading secrets/<component>/<name> through vault-backend, **refreshInterval 15s**; charts use existingSecret.
Exposure	Certificate (letsencrypt-prod-contour) + HTTPProxy with annotation projectcontour.io/ingress.class: contour-external and permitInsecure: false, both Objects; OPA and the route pattern of section 7.6 for anything that is not public.

© Terradue 2026 - The copyright in this document is vested in Terradue. This document may only be reproduced in whole or in part, or stored in a retrieval system, or transmitted in any form, or by any means electronic, mechanical, photocopying or otherwise, either with the prior permission of Terradue or in accordance with the terms of ESA Contract No. 4000144570/24/I-KE.

Rule	Detail
Placement	nodeSelector {aig/node-pool: static} in every chart key that sets it.
Versions	chart.version exactly as in the chart's Chart.yaml (no added v prefix); pinned image tags.
Private charts	chart.pullSecretRef: {name: …} only — the Secret must be in namespace platform (namespaced Releases cannot reference another namespace).
Drift detection	enableDriftDetection: true by default; false for operator charts that mutate their own objects and for charts that render namespaced objects without metadata.namespace (section 6.3).
Data holders	argocd.argoproj.io/sync-options: Prune=false on Releases/Objects that hold data (Vault, Kafka, KnativeServing, Argo releases); Objects of operator CRs without Delete in managementPolicies.
Metadata	labels ops.copernicuslac.io/component: <layer>, ops.copernicuslac.io/resource-name: <name> (≤ 63 characters), ops.copernicuslac.io/provider: ifxcloud; annotations ops.copernicuslac.io/source-path: <path>, ops.copernicuslac.io/resource-type: <apiVersion/Kind or helm-release>.
Change size	One commit, one component; an upgrade changes one Release at a time.

Example — a Certificate wrapped in an Object (platform/20-identity/keycloak/iam-tls.yaml):

```
apiVersion: kubernetes.m.crossplane.io/v1alpha1
kind: Object
metadata:
  name: iam-tls
  namespace: platform
  labels:
    ops.copernicuslac.io/component: 20-identity
    ops.copernicuslac.io/resource-name: iam-tls
    ops.copernicuslac.io/provider: ifxcloud
  annotations:
    ops.copernicuslac.io/source-path: platform/20-identity/keycloak/iam-tls.yaml
    ops.copernicuslac.io/resource-type: cert-manager.io/v1/Certificate
    argocd.argoproj.io/sync-wave: "2"
spec:
  forProvider:
    manifest:
      apiVersion: cert-manager.io/v1
      kind: Certificate
      metadata: {name: iam-tls, namespace: iam}
      spec:
        secretName: iam-tls
        dnsNames: [iam.copernicuslac.aig.gob.pa]
        issuerRef: {kind: ClusterIssuer, name: letsencrypt-prod-contour}
        privateKey: {algorithm: ECDSA, size: 256, rotationPolicy: Always}
  providerConfigRef: {name: kubernetes-provider, kind: ProviderConfig}
```

Example — the skeleton of a Release:

```
apiVersion: helm.m.terradue.crossplane.io/v1beta1
kind: Release
metadata:
  name: opa
  namespace: platform
  labels: {...}                          # as above
  annotations: {argocd.argoproj.io/sync-wave: "1", ...}
spec:
  forProvider:
    enableDriftDetection: true
    wait: true
    waitTimeout: 5m
    chart:
      name: opa
      # chart from this platform's Harbor
      repository: oci://cr.copernicuslac.aig.gob.pa/opa-authz
```

© Terradue 2026 - The copyright in this document is vested in Terradue. This document may only be reproduced in whole or in part, or stored in a retrieval system, or transmitted in any form, or by any means electronic, mechanical, photocopying or otherwise, either with the prior permission of Terradue or in accordance with the terms of ESA Contract No. 4000144570/24/I-KE.

```
      version: "0.2.5"
      pullSecretRef: {name: helm-repo-harbor}                # Secret in namespace platform
    namespace: opa
    skipCreateNamespace: true                                      # namespace is its own
Object
    values: {...}
  providerConfigRef: {name: helm-provider, kind: ProviderConfig}
```

6.3 provider-helm Behaviour

The behaviours below were met during the deployment; each one has a fixed rule.

Table 37 – provider-helm behaviours and rules

Behaviour	Symptom	Rule
Polls a synced Release about every 10 minutes	Release Ready only after up to 10 min	Force a check: `kubectl -n platform annotate releases.helm.m.terradue.crossplane.io <name> reconcile-at="$(date +%s)" --overwrite`
With wait: true, new values are applied only after the running install reaches waitTimeout	A fix in Git (e.g. image tag) seems ignored	Wait for waitTimeout; a failed first install can be unblocked by deleting the Release in Argo CD and syncing again — **never** for charts that install CRDs (knative-operator, Strimzi, Argo)
Compares chart.version as a string	"v1.20.1" vs 1.20.1 → helm upgrade at every poll, Ready=False	Write the version exactly as in Chart.yaml
Drift detection on objects mutated by an operator	New Helm revision every ~10 min, Ready=False "Unavailable" (knative-operator, Argo Events EventSource/Sensor/EventBus)	enableDriftDetection: false
Drift detection on objects without metadata.namespace	"an empty namespace may not be set when a resource name is provided", Degraded (strimzi-kafka-operator)	enableDriftDetection: false
pullSecretRef of a namespaced Release has no namespace field	Schema error on namespace	Secret in namespace platform
Orphan CRDs from an earlier install (helm keep)	"CustomResourceDefinition … exists and cannot be imported into the current release: invalid ownership metadata"	Never delete the CRDs (it deletes every CR). Re-annotate them: meta.helm.sh/release-name=<new>, meta.helm.sh/release-namespace=<ns>, label app.kubernetes.io/managed-by=Helm, then reconcile-at

```
# re-assign orphan CRDs to a new release (example: Argo Workflows)
for c in $(kubectl get crd -o json | jq -r '.items[] |
    select(.metadata.annotations["meta.helm.sh/release-name"]=="<old>") | .metadata.name'); do
  kubectl annotate crd $c meta.helm.sh/release-name=argo-workflows \
    meta.helm.sh/release-namespace=processing --overwrite
  kubectl label crd $c app.kubernetes.io/managed-by=Helm --overwrite
done
```

© Terradue 2026 - The copyright in this document is vested in Terradue. This document may only be reproduced in whole or in part, or stored in a retrieval system, or transmitted in any form, or by any means electronic, mechanical, photocopying or otherwise, either with the prior permission of Terradue or in accordance with the terms of ESA Contract No. 4000144570/24/I-KE.

6.4 Adding a Component

1. Check the chart: download it into bootstrap/harbor/archives/charts/ (`helm pull … -d …`), read values.yaml and the templates, verify the Kubernetes 1.33 compatibility and the latest patch of the version line in use.
2. Prepare the external prerequisites: database and pg_hba (section 3.5.1), bucket and user (section 3.5.2), DNS, Keycloak client.
3. Write the secrets to Vault (section 7.1).
4. Create platform/<layer>/<component>/ from the template: namespace Object, ExternalSecrets, Release, Certificate, HTTPProxy, README with prerequisites and checks.
5. Validate locally: `python3 -c "import yaml,glob; [list(yaml.safe_load_all(open(f))) for f in glob.glob('platform/**/*.yaml', recursive=True)]"`; where possible `helm template` with the values of the Release.
6. Add the chart to harbor/seed.yaml and run ./bootstrap.sh harbor-seed; the Release uses oci://cr.copernicuslac.aig.gob.pa/<project> with pullSecretRef helm-repo-harbor (the project must be among those of robot$helm-pull).
7. Merge request, merge, follow the layer Application; check the Release (Synced, Ready), pods, Certificate and HTTPProxy.
8. Record decisions and issues in docs/deployment-notes.md.

© Terradue 2026 - The copyright in this document is vested in Terradue. This document may only be reproduced in whole or in part, or stored in a retrieval system, or transmitted in any form, or by any means electronic, mechanical, photocopying or otherwise, either with the prior permission of Terradue or in accordance with the terms of ESA Contract No. 4000144570/24/I-KE.

7 Security and Platform Foundation

Layers 10 to 30 and the security model of every public service. Every other layer depends on them for secrets, identity, authorisation or runtime.

7.1 Vault

Table 38 – Vault

Aspect	Value
Release	copernicuslac-vault — chart hashicorp/vault 0.34.1, Vault 2.0.4, Prune=false (platform/10-core/vault/)
Topology	HA with integrated storage (Raft), 3 pods; data and audit volumes 10 Gi each on longhorn
Namespace	secret-management; Service copernicuslac-vault-active points to the leader
Exposure	**Not exposed.** UI and CLI through `kubectl -n secret-management port-forward svc/copernicuslac-vault-active 8200:8200`
Readiness	probe /v1/sys/health?standbyok=true&sealedcode=204&uninitcode=204, so that sealed pods do not block the rollout; Vault is usable only when unsealed
Engines	KV v2 mounted at secrets/; Kubernetes auth; policy external-secrets-read (read-only on secrets/)
Unseal	Manual (no cloud KMS or HSM on this platform): every restarted pod comes back sealed

Table 39 – Vault operations (ops/vault/)

When	Script	Needs
First install, or rebuild with empty storage	init.sh	nothing: creates 5 unseal keys (threshold 3) and the root token
After init, any pod restart, node drain, cluster restart	unseal.sh	3 of the 5 unseal keys
After init or a rebuild, to restore the configuration	configure.sh	root token

IMPORTANT

init.sh writes the keys and the root token, SOPS-encrypted, to ops/vault/vault-init.sops.yaml (git-ignored). Store them in the team password manager (or split the keys between people), then delete the file from the workstation. Without the file the scripts ask for the keys and the token. The unseal keys are passed to the pods through stdin, never on the command line.

After a restart of any Vault pod, run ops/vault/unseal.sh: until then ExternalSecrets are not refreshed (existing Secrets keep working).

```
# expected: Sealed false, HA Mode active|standby
kubectl -n secret-management exec copernicuslac-vault-0 -- vault status
kubectl get clustersecretstore vault-backend                                  # READY True

# write a platform secret (values typed hidden, never in the shell history)
kubectl -n secret-management port-forward svc/copernicuslac-vault-active 8200:8200 \
  >/dev/null 2>&1 &
export VAULT_ADDR=http://localhost:8200; vault login
bash -c 'read -rsp "value: " V </dev/tty; echo
  vault kv put secrets/<component>/<name> <key>="$V"'
vault kv get -format=json secrets/<component>/<name> | jq -c '.data.data | keys'
```

© Terradue 2026 - The copyright in this document is vested in Terradue. This document may only be reproduced in whole or in part, or stored in a retrieval system, or transmitted in any form, or by any means electronic, mechanical, photocopying or otherwise, either with the prior permission of Terradue or in accordance with the terms of ESA Contract No. 4000144570/24/I-KE.

```
vault kv patch secrets/<component>/<name> <key>=<new value>    # change one key, keep the
others
```

7.2 External Secrets Operator

External Secrets Operator 2.13.0 (Release eso, namespace eso, 2 replicas of controller, webhook and cert-controller) copies values from Vault into Kubernetes Secrets and keeps them refreshed.

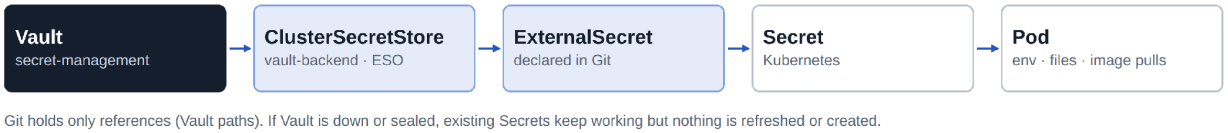


Figure 7 – Secret delivery chain

```
apiVersion: external-secrets.io/v1
kind: ClusterSecretStore
metadata:
  name: vault-backend
spec:
  conditions:
    - namespaceSelector:
        matchLabels: {eso: enabled}          # only labelled namespaces may use the store
  provider:
    vault:
      server: http://copernicuslac-vault-active.secret-management:8200
      path: secrets
      version: v2
      auth:
        kubernetes:
          mountPath: kubernetes
          role: external-secrets
          serviceAccountRef: {name: eso-external-secrets, namespace: eso}
```

- Every ExternalSecret has **refreshInterval: 15s**: a value changed in Vault reaches the Kubernetes Secret within 15 seconds. Pods that read the Secret as environment variables see the new value only after `kubectl -n <ns> rollout restart deploy/<name>`.
- Force an immediate refresh: `kubectl -n <ns> annotate externalsecret <name> force-sync="$(date +%s)" --overwrite`.
- If the store was validated before Vault was configured it stays not ready: the same annotation on the ClusterSecretStore forces a new validation.
- The inventory of Vault paths and ExternalSecrets is in Annex C.

7.3 TLS and Exposure Pattern

cert-manager and Contour are Tier 1 (section 5.3); Tier 3 components only declare a Certificate and an HTTPProxy as Objects. The Keycloak HTTPProxy shows the rules for administrative paths:

```
spec:
  virtualhost:
    fqdn: iam.copernicuslac.aig.gob.pa
    tls: {secretName: iam-tls, minimumProtocolVersion: "1.2"}
  routes:
    - conditions: [{prefix: /admin}]              # admin console: never on the Internet
      directResponsePolicy: {statusCode: 404}
    - conditions: [{prefix: /realms/master}]      # master realm: never on the Internet
```

© Terradue 2026 - The copyright in this document is vested in Terradue. This document may only be reproduced in whole or in part, or stored in a retrieval system, or transmitted in any form, or by any means electronic, mechanical, photocopying or otherwise, either with the prior permission of Terradue or in accordance with the terms of ESA Contract No. 4000144570/24/I-KE.

```
      directResponsePolicy: {statusCode: 404}
    - conditions: [{prefix: /}]
      permitInsecure: false
      services: [{name: iam-keycloakx-http, port: 80}]
      responseHeadersPolicy:
        set: [{name: Strict-Transport-Security, value: "max-age=31536000; includeSubDomains"}]
```

Table 40 – Services that are not published (port-forward only)

Service	Command	URL
Argo CD	kubectl -n argocd port-forward svc/argocd-server 8080:80	http://localhost:8080
Vault	kubectl -n secret-management port-forward svc/copernicuslac-vault-active 8200:8200	http://localhost:8200
Keycloak admin console	kubectl -n iam port-forward svc/iam-keycloakx-http 8080:80	http://localhost:8080/admin
Kafka UI	kubectl -n kafka-ui port-forward svc/copernicuslac-kafka-ui 8080:80	http://localhost:8080
Argo Workflows UI	kubectl -n processing port-forward svc/argo-workflows-server 2746:2746	http://localhost:2746
GitLab, Harbor (bootstrap / recovery only)	svc/copernicuslac-gitlab-webservice-default 8181:8181 · svc/harbor 8282:80	http://localhost:8181 · :8282

7.4 Keycloak

Table 41 – Keycloak

Aspect	Value
Release	iam — chart codecentric keycloakx 7.4.0, Keycloak 26.8.1, 2 replicas, clustering through the database (jdbc-ping)
Database	keycloak on the managed PostgreSQL, role keycloak CONNECTION LIMIT 30, password from Vault secrets/iam/keycloak-external-db; pool --db-pool-max-size=10 per replica
Host	https://iam.copernicuslac.aig.gob.pa (--hostname, --hostname-strict=true); admin console on --hostname-admin=http://localhost:8080 through port-forward
Themes	PVC 1 Gi RWX on longhorn (keycloak-themes-pvc)
Management	/health and /metrics on port 9000, cluster-internal only

Prerequisites (from platform/20-identity/keycloak/README.md):

```
-- pgAdmin, on the primary, one statement at a time
CREATE ROLE keycloak WITH LOGIN PASSWORD '<password>' CONNECTION LIMIT 30;
GRANT keycloak TO CURRENT_USER;
CREATE DATABASE keycloak OWNER keycloak ENCODING 'UTF8' TEMPLATE template0;
REVOKE ALL ON DATABASE keycloak FROM PUBLIC;
-- pg_hba.conf on both nodes:  host keycloak keycloak 192.168.80.0/24 scram-sha-256

vault kv put secrets/iam/keycloak-external-db password='<same password>'
vault kv put secrets/iam/keycloak-admin username=bootstrap-admin \
  password="$(openssl rand -base64 24)"
```

After the first start:

© Terradue 2026 - The copyright in this document is vested in Terradue. This document may only be reproduced in whole or in part, or stored in a retrieval system, or transmitted in any form, or by any means electronic, mechanical, photocopying or otherwise, either with the prior permission of Terradue or in accordance with the terms of ESA Contract No. 4000144570/24/I-KE.

1. Log in as the bootstrap admin through the port-forward, create personal administrators with OTP in the master realm, then **delete the bootstrap admin** (the ExternalSecret and the KC_BOOTSTRAP_* variables are removed from the Release; a comment in iam.yaml explains how to bring them back for a new install).
2. Set the Frontend URL of the master realm to http://localhost:8080 (with kcadm in the pod): /realms/master is not published and without it the admin console fails with "Timeout when waiting for 3rd party check iframe message".
3. Create and harden the platform realm (table below). A partial export is kept in ops/keycloak/CopernicusLAC-Platform-realm.json.

Table 42 – Realm CopernicusLAC-Platform

Setting	Value
Issuer	https://iam.copernicuslac.aig.gob.pa/realms/CopernicusLAC-Platform
Registration	No self-registration; no forgot-password and no verify-email until an SMTP relay is configured [TBC-7]
Second factor	OTP mandatory for every user: browser flow "Browser - Conditional 2FA" and "OTP Form" set to Required
Brute force	Detection on, lock after 5 failures
Password policy	Length 12, upper case, digit, special character, not username, not e-mail, not recently used (5)
Sessions and tokens	SSO idle 30 min, SSO max 10 h, access token 5 min
Events	User and admin events stored 30 days
Clients	gitlab (configured), discover (oauth2-proxy of the catalog), argo-workflows (prepared), processing clients processing-coplac / results-coplac, stac-browser, Portal, App Hub, Drive (to create) — redirect URIs without wildcards
Groups	main, sysadmin, admin, Operator, writers-processing and one group per namespace of processing (to create, section 12.2); groups claim with plain names (Group Membership mapper, full path OFF) in the ID token

7.5 OPA

Table 43 – OPA

Aspect	Value
Release	opa — chart opa 0.2.5 (Terradue, from oci://cr.copernicuslac.aig.gob.pa/opa-authz), image openpolicyagent/opa:1.21.2-envoy-3, 2 replicas, PodDisruptionBudget minAvailable 1, read-only root filesystem
Integration	Envoy external authorisation over gRPC 9191; Contour ExtensionService opa/opa-authz; every HTTPProxy that sets virtualhost.authorization.extensionRef {name: opa-authz, namespace: opa} is authorised by OPA
Policy	files/policy.rego of the chart (package envoy.authz, query data.envoy.authz.result) — not overridable by values
Image tags	The -envoy tags follow the opa-envoy-plugin releases (1.21.2-envoy-3), not the OPA releases: 1.21.3-envoy does not exist

Table 44 – Policy data (values data.authz)

Key	Panama value
iss / aud / azp	realm CopernicusLAC-Platform issuer / account / discover

© Terradue 2026 - The copyright in this document is vested in Terradue. This document may only be reproduced in whole or in part, or stored in a retrieval system, or transmitted in any form, or by any means electronic, mechanical, photocopying or otherwise, either with the prior permission of Terradue or in accordance with the terms of ESA Contract No. 4000144570/24/I-KE.

Key	Panama value
cert_pem	public signing certificate of the realm (RS256, use=sig) — changes when the realm key is rotated: `curl -s <issuer>/protocol/openid-connect/certs | yq -p json '.keys[] | select(.alg == "RS256" and .use == "sig") | .x5c[0]'`
processing_host / processing_allowed_clients	processing.copernicuslac.aig.gob.pa / results-coplac, discover, processing-coplac
catalog_hosts / catalog_allowed_clients	results., discover. / stac-browser
titiler_hosts, download_host	view., download.
admin bypass	enable_admin_bypass true, admin_groups admin, sysadmin

Table 45 – Decision of the policy by host (chart 0.2.5)

Host type	Without Authorization header	With a valid Bearer token
processing	Only OPTIONS (CORS preflight); everything else 403	azp/aud in processing_allowed_clients; namespace (first path segment) in the user groups; writes need writers-processing and write permission on the namespace
catalog (discover, results)	**Any method allowed** — designed to be followed by oauth2-proxy (section 7.6)	Operator group: all methods; otherwise audience/azp check, namespace group, write permission for writes
download	GET and OPTIONS (pass-through to oauth2-proxy)	GET with processing audience and namespace access; PUT for writers-processing with write permission
other hosts (e.g. view)	Allowed (pass-through to oauth2-proxy)	Audience check; TiTiler GET needs a processing client

7.6 Protecting a Public Service

A public HTTPProxy that serves data to authenticated users follows one pattern, used today by the catalog discover. It combines OPA (API clients with a Bearer token) and oauth2-proxy (browser users).

```
virtualhost:                        # root HTTPProxy
  fqdn: discover.copernicuslac.aig.gob.pa
  tls: {secretName: discover-tls}
  authorization:
    extensionRef: {name: opa-authz, namespace: opa}
routes:                             # child HTTPProxy of the workload (/main)
  - conditions:                     # 1. a REAL Bearer token → API, decided by OPA
      - prefix: /
      - header: {name: authorization, regex: '(?i)bearer\s+\S+'}
    services: [{name: nginx-stripper-catalog-main, port: 80}]
  - authPolicy: {disabled: true}    # 2. oauth2-proxy endpoints (login callback)
    conditions: [{prefix: /oauth2}]
    services: [{name: oauth2-proxy-main, port: 80}]
  - conditions: [{prefix: /}]       # 3. everything else → oauth2-proxy (login), NEVER the API
    services: [{name: oauth2-proxy-main, port: 80}]
```

Table 46 – oauth2-proxy settings

Setting	Value
Image	quay.io/oauth2-proxy/oauth2-proxy:v7.12.0, provider keycloak-oidc, one Deployment per workload

© Terradue 2026 - The copyright in this document is vested in Terradue. This document may only be reproduced in whole or in part, or stored in a retrieval system, or transmitted in any form, or by any means electronic, mechanical, photocopying or otherwise, either with the prior permission of Terradue or in accordance with the terms of ESA Contract No. 4000144570/24/I-KE.

Setting	Value
Client	Confidential Keycloak client (discover for the catalog main), standard flow only, redirect https://<host>/<ns>/oauth2/callback
Groups	allowed groups <ns>,sysadmin; the groups claim must be in the ID token with plain names (mapper in a Default client scope; oauth2-proxy requests only scope openid)
Secrets	Vault secrets/catalog/oauth2-proxy: OAUTH2_PROXY_CLIENT_ID, OAUTH2_PROXY_CLIENT_SECRET, OAUTH2_PROXY_COOKIE_SECRET
Cookie secret	Must be 16, 24 or 32 bytes: `openssl rand -hex 16` (32 characters). Otherwise: "cookie_secret must be 16, 24, or 32 bytes to create an AES cipher"
Writes	With a token: as decided by OPA. With a browser session: any member of the allowed groups (OPA is not in that path) — the allowed groups are therefore the owners of the catalog

7.7 Knative Serving

Table 47 – Knative

Aspect	Value
Operator	Release knative-operator — chart knative-operator 1.20.1 (https://knative.github.io/operator), enableDriftDetection false (the operator patches its own webhook certificates and CRDs), Prune=false
Serving	Object KnativeServing 1.20.1 in knative-serving: high availability 2, ingress contour, config-network ingress-class contour.ingress.networking.knative.dev, feature **kubernetes.podspec-nodeselector: enabled** (required by the processing functions' nodeSelector), workloads activator, autoscaler, autoscaler-hpa, controller, webhook, net-contour-controller on the static pool; managementPolicies without Delete, readiness DeriveFromObject, Prune=false
Why the operator	The release manifests of Knative are generated assets, not files of a Git repository; the operator is installed by a chart and takes one small KnativeServing resource
Upgrade	One minor at a time: Contour targetRevision (Tier 1), operator chart, then KnativeServing spec.version

Check: `kubectl -n knative-serving get knativeserving` (Ready True) and `kubectl get ksvc -A`. Knative creates HTTPProxies named <function>-contour-internal-* for each service; they are cluster-local and expected.

© Terradue 2026 - The copyright in this document is vested in Terradue. This document may only be reproduced in whole or in part, or stored in a retrieval system, or transmitted in any form, or by any means electronic, mechanical, photocopying or otherwise, either with the prior permission of Terradue or in accordance with the terms of ESA Contract No. 4000144570/24/I-KE.

8 Data Services

Layer 40 (platform/40-data/). Kafka and the STAC API are deployed; TiTiler, the Geobrowser, STAC Browser and the results catalogs are to be deployed (section 12.1).

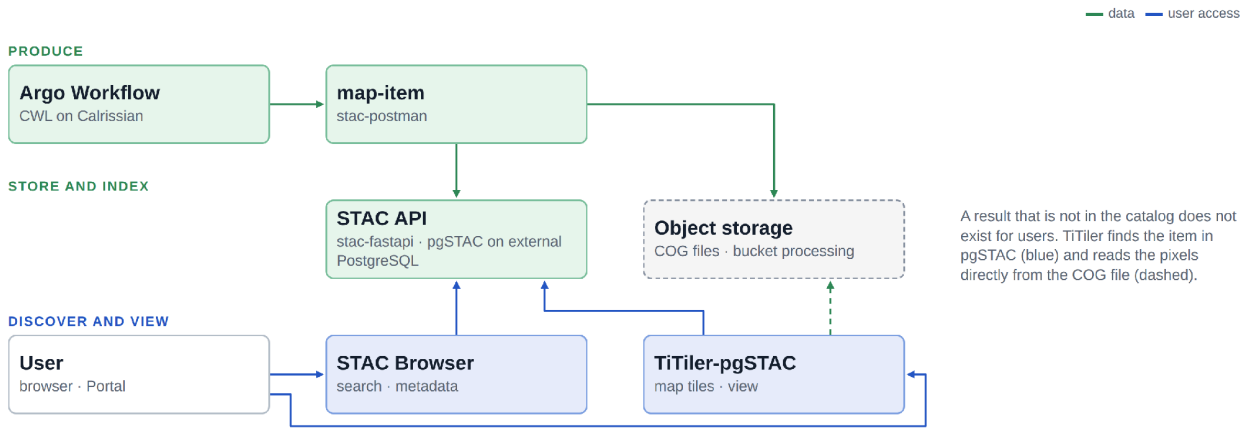


Figure 8 – From a processing run to the catalog and the map (functional)

8.1 Kafka

Table 48 – Kafka

Aspect	Value
Operator	Release copernicuslac-strimzi-operator — chart strimzi-kafka-operator 1.1.0 (oci://quay.io/strimzi-helm), namespace kafka-operator, watching only namespace kafka, 2 replicas with leader election and PodDisruptionBudget, enableDriftDetection false
Cluster	copernicuslac-kafka, Kafka 4.2.1, metadataVersion 4.2-IV1, KRaft (Strimzi 1.x API kafka.strimzi.io/v1)
Nodes	KafkaNodePool dual-role: 3 × (controller + broker), one per node (pod anti-affinity), static pool, 20 Gi each on **longhorn-processing**, deleteClaim false, JVM heap 1 Gi
Listeners	plain 9092 and tls 9093, internal only; bootstrap copernicuslac-kafka-kafka-bootstrap.kafka:9092
Durability	default.replication.factor 3, min.insync.replicas 2 — one node can be down without losing writes
Topics	**auto.create.topics.enable true, num.partitions 3** (decision 2026-10-23, see below); topics with specific settings declared as KafkaTopic
Protection	Kafka Object Prune=false, managementPolicies without Delete, readiness DeriveFromObject

NOTE

Why topic auto-creation is enabled. The CloudEvents Sensor publishes each event to the topic named in the X-Kafka-Topic header sent by the processing API; the names depend on the process and are not known in advance.

- Restarts and upgrades go through the Kafka resource; never delete broker pods in bulk or edit the StrimziPodSets.
- Strimzi installs its CRDs only at the first install of the chart: apply the new CRDs before every Strimzi upgrade, then upgrade the operator, then spec.kafka.version.

© Terradue 2026 - The copyright in this document is vested in Terradue. This document may only be reproduced in whole or in part, or stored in a retrieval system, or transmitted in any form, or by any means electronic, mechanical, photocopying or otherwise, either with the prior permission of Terradue or in accordance with the terms of ESA Contract No. 4000144570/24/I-KE.

- The plain listener has no authentication: any pod can produce and consume. Target: SCRAM and authorisation on the TLS listener, one KafkaUser per service with its password in Vault, then remove the plain listener.
- Check: `kubectl -n kafka get kafka,kafkanodepool,kafkatopic` (Ready True).

8.2 Kafka UI

Release copernicuslac-kafka-ui — chart kafka-ui 1.4.2 (kafbat, from oci://cr.copernicuslac.aig.gob.pa/strimzi-kafka), namespace kafka-ui, 1 replica, cluster copernicuslac-kafka through the Strimzi bootstrap Service.

Access: `kubectl -n kafka-ui port-forward svc/copernicuslac-kafka-ui 8080:80`.

8.3 STAC API

Table 49 – STAC API (catalog main)

Aspect	Value
Release	copernicuslac-stac-api — chart stac-fastapi-pgstac 2.0.7 (from oci://cr.copernicuslac.aig.gob.pa/stac-utils), image ghcr.io/stac-utils/stac-fastapi-pgstac:6.2.1 (public), Prune=false
Namespace	catalog (workload key catalog; label eso=enabled)
Public URL	https://discover.copernicuslac.aig.gob.pa/main (Certificate discover-tls); root HTTPProxy stac-fastapi-main-root with OPA, child HTTPProxy stac-fastapi-child-main with the 3-route pattern of section 7.6
Containers	stac-fastapi-pgstac (uvicorn with --root-path /main and proxy headers) + nginx stripper 1.28-alpine that removes /main (Service nginx-stripper-catalog-main)
Database	pgstac on the managed PostgreSQL, **pgSTAC 0.9.12**; credentials Vault secrets/catalog/pgstac (username, password) → Secret pgstac-secret (POSTGRES_USER, POSTGRES_PASS)
Transactions	enable_transactions true: writes only with a token decided by OPA, or with a browser session of a member of main / sysadmin
Login	oauth2-proxy-main, client discover, allowed groups main,sysadmin, secrets Vault secrets/catalog/oauth2-proxy
Connections	**4 uvicorn workers × pool 10 = 40 (web_concurrency 4, db_max_conn_size 10)**
Other settings	reload false , use_api_hydrate true, GDAL HTTP tuning, static pool, restricted securityContext

Database procedure (pgAdmin on nodo01 with the DBA account; PostGIS, btree_gist and unaccent are available in template1):

1. Generate the password and write it to Vault first: `vault kv put secrets/catalog/pgstac username=pgstac password="$P"`.
2. On database postgres, one statement at a time: `CREATE ROLE pgstac LOGIN PASSWORD '<paste>' CONNECTION LIMIT 50;` · `CREATE DATABASE pgstac;` (owner: the DBA account, never the application user) · `REVOKE ALL ON DATABASE pgstac FROM PUBLIC;` · `GRANT CONNECT ON DATABASE pgstac TO pgstac;`.
3. On a first install create the pgSTAC roles so that they can be granted before the script: `CREATE ROLE pgstac_admin; CREATE ROLE pgstac_read; CREATE ROLE pgstac_ingest;`, then `GRANT CONNECT, TEMPORARY ON DATABASE pgstac TO pgstac_admin, pgstac_read, pgstac_ingest;` — without TEMPORARY the script fails with 42501 "permission denied to create temporary tables" (the REVOKE also removes TEMPORARY).

© Terradue 2026 - The copyright in this document is vested in Terradue. This document may only be reproduced in whole or in part, or stored in a retrieval system, or transmitted in any form, or by any means electronic, mechanical, photocopying or otherwise, either with the prior permission of Terradue or in accordance with the terms of ESA Contract No. 4000144570/24/I-KE.

4. On database pgstac run https://raw.githubusercontent.com/stac-utils/pgstac/v0.9.12/src/pgstac/pgstac.sql (extensions, roles, schema pgstac owned by pgstac_admin, migrations table).
5. The script leaves the session with SET ROLE: run `RESET ROLE;` (or open a new Query Tool) before the next statements, otherwise they fail with 42501 ("must be owner", "only roles with ADMIN option").
6. `GRANT pgstac_ingest TO pgstac;` (read and write, not pgstac_admin: schema upgrades are done by the operator with a new pgstac.sql) · `ALTER ROLE pgstac SET search_path TO pgstac, public;` · `SELECT pgstac.get_version();` → 0.9.12.
7. pg_hba line `host pgstac pgstac 192.168.80.0/24 scram-sha-256` on both nodes and reload.

8.4 TiTiler-pgSTAC (to deploy)

TiTiler renders map tiles from the COGs referenced by the STAC items. The planned configuration will be:

Table 50 – TiTiler — planned configuration

Aspect	Panama
Namespace	catalog (reuses pgstac-secret of the STAC API)
Host	view.copernicuslac.aig.gob.pa/main + Certificate view-tls
Requests without token	oauth2-proxy, groups main,sysadmin; route 1 with the Bearer regex
Database connections	**2 workers × pool 2 = 4**
Tile cache	Proposed: 2 GB, PVC 10 Gi longhorn, 2–3 Gi RAM; check the Redis image of the chart (Bitnami images are no longer public)
Object storage	StorageGRID, path-style, read-only user on the result buckets
Placement	aig/node-pool: static

8.5 Geobrowser (to deploy)

The Geobrowser (glowstac) is the map client of the platform.No OPA on the host: the application signs users in with Keycloak in the browser.

Table 51 – Corrections to the runtime configuration (config.dat)

Key	Panama
stacApiEndpoint	https://discover.copernicuslac.aig.gob.pa/main
stacApiProtectedList	"https://discover.copernicuslac.aig.gob.pa"
auth.clientId	processing-coplac, or add processing to OPA (decision pending)
Keycloak client	public client, redirect https://<host>/main/*, web origin https://<host>
Other endpoints	kept; active when the services exist

© Terradue 2026 - The copyright in this document is vested in Terradue. This document may only be reproduced in whole or in part, or stored in a retrieval system, or transmitted in any form, or by any means electronic, mechanical, photocopying or otherwise, either with the prior permission of Terradue or in accordance with the terms of ESA Contract No. 4000144570/24/I-KE.

8.6 STAC Browser (to deploy)

- **STAC Browser**: static front end; if exposed on a catalog host it follows the same OPA rules (client stac-browser).

© Terradue 2026 - The copyright in this document is vested in Terradue. This document may only be reproduced in whole or in part, or stored in a retrieval system, or transmitted in any form, or by any means electronic, mechanical, photocopying or otherwise, either with the prior permission of Terradue or in accordance with the terms of ESA Contract No. 4000144570/24/I-KE.

9 Processing Services

Layer 50 (platform/50-processing/) and layer 70 (application packages). Workflows run in namespace **processing**; the processing API runs in namespace **mate-api**. The first path segment of the API is the namespace of the workflows: https://processing.copernicuslac.aig.gob.pa/processing/….

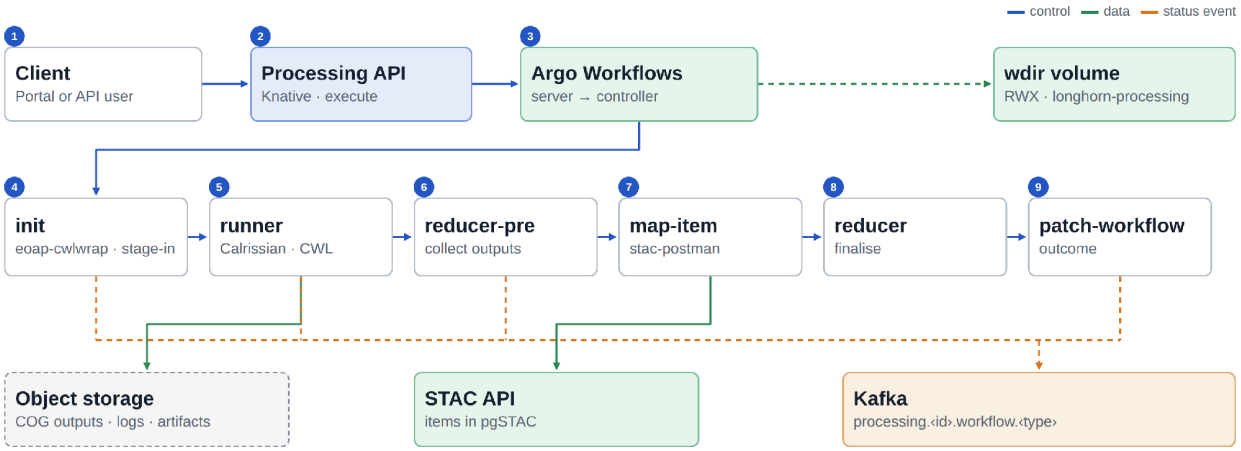


Figure 9 – One processing request, end to end (functional; the CWL runner is still to be deployed)

9.1 Argo Workflows

Table 52 – Argo Workflows

Aspect	Value
Release	argo-workflows — chart argo-workflows 0.45.26 (Argo Workflows v3.7.2), Prune=false. The name is fixed: the processing API reaches the server as argo-workflows-server.<namespace>:2746
Scope	singleNamespace (processing); CRDs installed and kept; aggregate roles; clusterWorkflowTemplates enabled
Controller	2 replicas (leader election), static pool; workflow defaults: serviceAccountName argo-sa, ttlStrategy 1800 s after completion, podGC deleteDelayDuration 5 s, artifactGC OnWorkflowDeletion with argo-sa
Artifacts	StorageGRID bucket copernicuslac-argo-artifacts (svr-cop-s3-pan.ifxcloud.com:10443, path-style), logs archived; key of user argowf from Vault secrets/processing/argo-artifacts (accesskey, secretkey) → argo-artifacts-creds
Archive	PostgreSQL database argowf on 10.0.0.3, table argo_workflows, archiveTTL 90 days, nodeStatusOffLoad; credentials Vault secrets/processing/argo-postgres → argo-postgres-config; pool 2 idle / 10 open. Argo creates its tables at the first start.
Server	2 replicas; authModes **server + client** (TEMPORARY, section 12.3); UI through port-forward only
Identity of workflows	ServiceAccount argo-sa with Role argo-sa-role (pods, logs, PVCs, ConfigMaps, jobs, workflows, templates, cron workflows, leases, task results, artifact GC tasks);long-lived token Secret argo-sa.service-account-token used by the processing API

© Terradue 2026 - The copyright in this document is vested in Terradue. This document may only be reproduced in whole or in part, or stored in a retrieval system, or transmitted in any form, or by any means electronic, mechanical, photocopying or otherwise, either with the prior permission of Terradue or in accordance with the terms of ESA Contract No. 4000144570/24/I-KE.

Prerequisites: database argowf (role argowf, CONNECTION LIMIT 25) created with the procedure of section 3.5.1; bucket copernicuslac-argo-artifacts and user argowf with ops/storagegrid/argowf-group-policy.json (list and location on the bucket; get, put, delete and multipart on its objects only); Vault secrets.

```
kubectl -n processing get externalsecret                    # SecretSynced
kubectl -n processing logs deploy/argo-workflows-workflow-controller \
  | grep -iE 'persist|archive|migrat|error' | head
kubectl -n processing port-forward svc/argo-workflows-server 2746:2746     # UI
```

9.2 Argo Events, Event Bus and Webhook

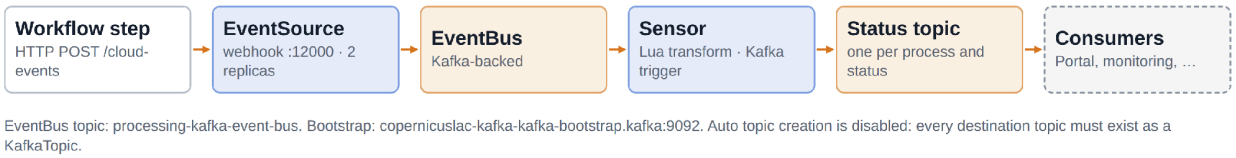


Figure 10 – Workflow status events

Table 53 – Event components

Release	Chart	Creates
argo-events	argo-events 2.4.27 (argo-helm)	Argo Events controller restricted to namespace processing, 2 replicas, CRDs kept
kafka-event-bus	kafka-event-bus 0.1.0 (seda, from oci://cr.copernicuslac.aig.gob.pa/seda)	EventBus kafka-event-bus on copernicuslac-kafka-kafka-bootstrap.kafka:9092
cloud-event-webhook	cloud-event-webhook 0.3.0 (seda, from oci://cr.copernicuslac.aig.gob.pa/seda)	EventSource cloud-events-webhook (Service cloud-events-webhook-eventsource-svc:12000, path /cloud-events) and Sensor cloud-events-webhook-sensor with a Kafka trigger; ServiceAccount argo-sa; debug logging off

- Both seda Releases have enableDriftDetection false: the Argo Events controller adds defaults to the EventBus, EventSource and Sensor, and drift detection would upgrade them every 10 minutes (Release Ready=False "Unavailable").
- Argo Events names its topics <namespace>-<eventbus>[-<sensor>-trigger|-action]. They are declared as KafkaTopic in platform/50-processing/event-bus/kafka-topics.yaml (3 partitions, replication 3, 7 days): processing-kafka-event-bus, processing-kafka-event-bus-cloud-events-webhook-sensor-trigger and -action. **Every new Sensor on this EventBus needs its own -trigger and -action topics.**
- The Object names of the topics are shortened (kafka-topic-webhook-sensor-trigger …): a label value cannot exceed 63 characters; the full topic name stays in the manifest.
- The Sensor reads the destination topic from the X-Kafka-Topic header and publishes the CloudEvent to it (auto-created, section 8.1).
- Check: `kubectl -n processing get eventbus,eventsource,sensor` and the Sensor log line "Consuming".

© Terradue 2026 - The copyright in this document is vested in Terradue. This document may only be reproduced in whole or in part, or stored in a retrieval system, or transmitted in any form, or by any means electronic, mechanical, photocopying or otherwise, either with the prior permission of Terradue or in accordance with the terms of ESA Contract No. 4000144570/24/I-KE.

9.3 Processing API (mate-api)

Table 54 – Processing API

Aspect	Value
Release	mate-api — chart mate-api 1.2.2 (from oci://cr.copernicuslac.aig.gob.pa/mate), namespace mate-api
Functions	10 Knative Services, images cr.terradue.com/mate/<function>:1.2.0 (TEMPORARY: copy to Harbor blocked, TBC-1), min 1 / max 10, port 8090: get-api, get-conformance-classes, get-processes, get-process-description, execute, get-jobs, get-status, get-result, get-logs, get-package
Exposure	Created by the chart: Certificate processing-tls (letsencrypt-prod-contour) and HTTPProxy mate-api-routing (contour-external, OPA authorisation, CORS, one regex route per function)
Identity	ServiceAccount mate-api-sa (pull secret cr-terradue-pull); it may read only the Secret argo-sa.service-account-token in processing (Role mate-api-token-reader)
Settings	ARGO_SERVER_HOST http://argo-workflows-server.{namespace}:2746, AUTH_TOKEN_NAME argo-sa.service-account-token, cloudEventsWebhook http://cloud-events-webhook-eventsource-svc.{namespace}.svc.cluster.local:12000/cloud-events, operatorProcessingSecretName operator-processing (optional, read by execute)
Security context	non-root (1001:100), no privilege escalation, all capabilities dropped, RuntimeDefault seccomp
Placement	static pool — requires the Knative feature kubernetes.podspec-nodeselector (section 7.7)

9.4 CWL Runner and Application Packages (to deploy)

- **CWL runner templates** (standard WorkflowTemplates for stage-in, Calrissian execution, STAC publication, status events): removed from the first delivery; to be deployed in layer 50 with images in Harbor and the S3, registry and operator-processing secrets in Vault.
- **Application packages** (layer 70): the composition calrimate (function-calrimate in Crossplane) turns one ApplicationPackage into the WorkflowTemplate, the CWL ConfigMap and the semaphore of a process. Prerequisites: the function package in Harbor, the CWL runner, the Kafka topics or auto-creation (in place).

© Terradue 2026 - The copyright in this document is vested in Terradue. This document may only be reproduced in whole or in part, or stored in a retrieval system, or transmitted in any form, or by any means electronic, mechanical, photocopying or otherwise, either with the prior permission of Terradue or in accordance with the terms of ESA Contract No. 4000144570/24/I-KE.

10 User-facing Services

10.1 Portal, App Hub and Drive (to deploy)

Layer 60 is still empty. The functional description of issue 2.0 (Portal: landing page, CMS, marketplace, back office; App Hub: JupyterHub with profiles generated by the Portal; Drive: shared files on StorageGRID) remains valid. Each service follows the procedure of section 6.4; the corrections already identified are:

- every service on the managed PostgreSQL with its own role, CONNECTION LIMIT and pg_hba lines;
- Drive: StorageClass longhorn instead of the removed longhorn-perf-r1, probes enabled, SMTP relay or mail disabled, HTTPProxy with the ingress-class annotation, no POSTGRES_SSLMODE override;
- App Hub: OAuth against the realm CopernicusLAC-Platform (not an external development realm), drift loop of the Release checked before adoption;
- Portal: no NodePort Service; front ends reachable only through Envoy.

© Terradue 2026 - The copyright in this document is vested in Terradue. This document may only be reproduced in whole or in part, or stored in a retrieval system, or transmitted in any form, or by any means electronic, mechanical, photocopying or otherwise, either with the prior permission of Terradue or in accordance with the terms of ESA Contract No. 4000144570/24/I-KE.

11 Operations

11.1 Shipping a Change

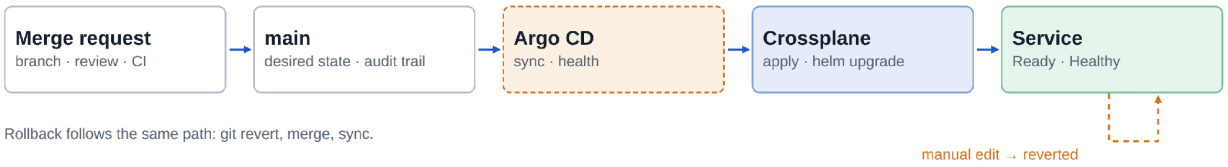


Figure 11 – Change workflow (Tier 3)

Table 55 – Change workflow by tier

Tier	Change	Apply	Verify
3	Merge request on platform/ (one component)	Merge: Argo CD syncs the layer within 3 minutes; Crossplane applies	Layer Application Synced/Healthy; Release Ready; service answers
2	Merge request on bootstrap/ (helmfile values)	helmfile -l name=<release> diff, then apply	Providers Healthy
1	Merge request on bootstrap/<component>/	kubectl apply -f the Application; argocd app diff; argocd app sync (or the bootstrap.sh step)	Application Synced/Healthy
Secrets	Vault (Tier 3) or sops (Tiers 1–2)	ESO refresh within 15 s / ./bootstrap.sh secrets	Rollout restart of the pods that read the Secret as environment

Rollback follows the same path: git revert, merge, and both loops converge on the previous state. Charts with data migrations (GitLab, Keycloak, Harbor) need a backup before the upgrade.

11.2 Daily Health Check

Run the checks bottom-up; a failure at one line can explain everything below it.

Table 56 – Daily health check

#	Check	Command	Expected
1	Nodes	kubectl get nodes	all Ready
2	Storage	kubectl -n longhorn-system get volumes.longhorn.io	robustness healthy
3	Tiers 1–2	cd bootstrap && ./bootstrap.sh status	all Synced/Healthy, providers Healthy
4	Vault	kubectl -n secret-management get pods · vault status	3/3 Running, Sealed false
5	Secrets chain	kubectl get clustersecretstore vault-backend · kubectl get externalsecret -A	Ready · SecretSynced
6	Tier 3 delivery	kubectl -n argocd get applications	Synced, Healthy
7	Crossplane resources	kubectl -n platform get releases.helm.m.terradue.crossplane.io,objects.kubernetes.m.crossplane.io	SYNCED True, READY True

© Terradue 2026 - The copyright in this document is vested in Terradue. This document may only be reproduced in whole or in part, or stored in a retrieval system, or transmitted in any form, or by any means electronic, mechanical, photocopying or otherwise, either with the prior permission of Terradue or in accordance with the terms of ESA Contract No. 4000144570/24/I-KE.

#	Check	Command	Expected
8	Routes and TLS	kubectl get httpproxies,certificates -A	valid · Ready
9	Processing	kubectl -n mate-api get ksvc · kubectl -n processing get workflows,eventsource,sensor	READY True · no stuck workflow
10	Kafka	kubectl -n kafka get kafka,kafkatopic	Ready
11	Databases	SELECT count(*) FROM pg_stat_activity;	well below max_connections

11.3 Troubleshooting

Debug along the delivery chain: did Git reach the cluster (Argo CD), did Crossplane act (conditions of the Release or Object), did the workload start (pods, events). The status.conditions message of a Release usually contains the exact Helm error: `kubectl -n platform describe releases.helm.m.terradue.crossplane.io <name> | tail -30`.

Table 57 – Known errors and fixes

Symptom	Cause	Fix
GitLab migrations "could not find database"	pg_hba typo or line missing on one node; database postgres not allowed	Fix pg_hba on both nodes, reload, check pg_hba_file_rules
Services fail with "too many clients"	max_connections exhausted (e.g. GitLab HPA scaled Sidekiq to 8)	Pools within the connection budget (section 3.5.1)
Harbor token requests return 500	Token-service key in PKCS#8	Generate PKCS#1 (gen-harbor-secrets.sh)
Login to GitLab/Harbor UI fails over port-forward	Secure cookies on plain HTTP	Use https://git. and https://cr.
Crossplane package not installable after a copy	xpkg annotations lost (docker save, oci-archive)	Copy with skopeo dir:/docker:// --preserve-digests
Contour Application OutOfSync	CRD preserveUnknownFields	ignoreDifferences (in place)
vault operator unseal does not read "-"	CLI limitation	Pass the key through stdin with sh -c 'read -r k; vault operator unseal "$k"' (unseal.sh)
ClusterSecretStore not ready after Vault configuration	Validated before configure.sh	force-sync annotation on the store
Keycloak admin console "3rd party check iframe" timeout	/realms/master not published	Frontend URL of master = http://localhost:8080
OPA ImagePullBackOff	Tag x.y.z-envoy that does not exist	Use the opa-envoy-plugin tags (1.21.2-envoy-3)
Release fix not applied	wait: true, install running until waitTimeout	Wait, or delete the Release in Argo CD and sync (no CRD charts)
Release upgraded every 10 minutes, Ready=False	chart.version string mismatch, or drift on operator-mutated objects	Exact version; enableDriftDetection false
Release Degraded "an empty namespace may not be set…"	Drift detection on objects without namespace	enableDriftDetection false
"CustomResourceDefinition … cannot be imported"	Orphan CRDs of an earlier release	Re-annotate the CRDs (section 6.3)
Knative rejects a Service with nodeSelector	Feature podspec-nodeselector disabled	Enable it in KnativeServing (in place)

© Terradue 2026 - The copyright in this document is vested in Terradue. This document may only be reproduced in whole or in part, or stored in a retrieval system, or transmitted in any form, or by any means electronic, mechanical, photocopying or otherwise, either with the prior permission of Terradue or in accordance with the terms of ESA Contract No. 4000144570/24/I-KE.

Symptom	Cause	Fix
Namespace stuck Terminating	Leftover object with finalizer (e.g. endpoints after a manual Knative removal)	Find the object with kubectl api-resources --namespaced -o name | xargs kubectl get -n <ns>, remove its finalizer
Sensor fatal "Topics do not exist"	Argo Events topics not declared	KafkaTopics (in place) or auto-create
Object SyncFailed "must be no more than 63 characters"	Label value too long	Shorter Object name
pgSTAC script fails 42501 on temporary tables	REVOKE removed TEMPORARY	GRANT CONNECT, TEMPORARY to the pgSTAC roles
GRANT / ALTER fail 42501 after pgstac.sql	Session still SET ROLE	RESET ROLE
oauth2-proxy "cookie_secret must be 16, 24, or 32 bytes"	Wrong secret length	openssl rand -hex 16
Anonymous request reaches a protected API	Route without token not on oauth2-proxy, or "present: true" on route 1	Pattern of section 7.6
Manual change reverted	Expected (self-heal)	Make the change in Git
GitLab "422: Sign-in using CopernicusLAC auth failed … already been taken"	User created at the first sign-in, still pending approval	Approve the user (Admin → Users → Pending approval)
git push rejected "not allowed to push code to protected branches"	User not Maintainer of the project (instance admin is not enough)	Role Maintainer or Owner in the group copernicuslac, or merge request
Release keeps the old chart repository after a change of source	provider-helm upgrades only when chart version or values change	Expected: the new source is used at the next upgrade; check the pull with helm registry login + helm pull
Harbor push fails: "trying to reuse blob … 500 Internal Server Error"	StorageGRID 12.0.0 returns 403 instead of 404 on missing keys (registry log s3aws: AccessDenied)	StorageGRID upgrade (TBC-1); check with s3cmd info on a missing key (section 3.5.2)

© Terradue 2026 - The copyright in this document is vested in Terradue. This document may only be reproduced in whole or in part, or stored in a retrieval system, or transmitted in any form, or by any means electronic, mechanical, photocopying or otherwise, either with the prior permission of Terradue or in accordance with the terms of ESA Contract No. 4000144570/24/I-KE.

11.4 Backup and Recovery - TBV by IFX

11.5 Disaster Recovery - TBV by IFX

© Terradue 2026 - The copyright in this document is vested in Terradue. This document may only be reproduced in whole or in part, or stored in a retrieval system, or transmitted in any form, or by any means electronic, mechanical, photocopying or otherwise, either with the prior permission of Terradue or in accordance with the terms of ESA Contract No. 4000144570/24/I-KE.

12 Deployment Status and Open Points

12.1 Deployment Status

Status on 23 October 2026. "Next action" is what unblocks or completes the component.

Table 58 – Deployment status by component

Tier / layer	Component	Status	Next action
T1	Argo CD, cert-manager, Contour, Valkey ×2	Deployed	—
T1	GitLab	Deployed	Publish registry. (Certificate + HTTPProxy) if the GitLab container registry is needed
T1	Harbor	Deployed	Copy the mate-api images after the StorageGRID upgrade (TBC-1)
T2	Crossplane, providers	Deployed	—
10	Vault, ESO, ClusterSecretStore	Deployed	Raft snapshots
20	Keycloak	Deployed	Clients and groups of the services (processing, Portal…)
20	OPA	Deployed	—
30	Knative	Deployed	—
40	Strimzi, Kafka, Kafka UI	Deployed	—
40	STAC API discover/main	Deployed	Browser login test with groups
40	TiTiler-pgSTAC	To deploy	Chart, DNS view., S3 read-only user, Keycloak client, Vault secrets
40	Geobrowser	To deploy	Chart, DNS, Keycloak public client, client id decision (section 8.5)
40	STAC Browser, results catalogs (6)	To deploy	Route pattern of section 7.6
50	Argo Workflows, Argo Events, event bus, webhook	Deployed	SSO instead of server mode
50	Processing API	Deployed	End-to-end test with a token (Keycloak groups/clients)
50	CWL runner templates	To deploy	Images in Harbor, secrets in Vault
60	Portal, App Hub, Drive	To deploy	Databases, buckets, Keycloak clients, corrections of chapter 10
70	Composition calrimate, application packages	To deploy	function-calrimate package, CWL runner

© Terradue 2026 - The copyright in this document is vested in Terradue. This document may only be reproduced in whole or in part, or stored in a retrieval system, or transmitted in any form, or by any means electronic, mechanical, photocopying or otherwise, either with the prior permission of Terradue or in accordance with the terms of ESA Contract No. 4000144570/24/I-KE.

12.2 Open Points

Table 59 – Open points

Ref.	Point	Owner	Priority
TBC-1	StorageGRID 12.0.0 returns 403 instead of 404 on missing objects (NetApp known issue, fixed after 12.0.0.4): upgrade the grid; blocks image pushes to Harbor (section 3.5.2)	IFX	High
TBC-2	effective_cache_size 16 GB on nodo01 and 8 GB on nodo02: align both nodes (reload only)	IFX	Low
TBC-3	Keycloak groups and clients for processing (processing-coplac, results-coplac), catalog groups, Operator, writers-processing; end-to-end tests with real tokens	Platform team	High
TBC-4	Geobrowser: client id (processing-coplac vs adding processing to OPA), host name	Platform / processing team	Medium
TBC-5	TiTiler: buckets to read, Redis image and size	Platform team	Medium
TBC-6	Labels and taints of the processing and storage pools on the Rancher machine pools	Platform team	Medium
TBC-7	SMTP relay for Keycloak (forgot password, verify e-mail) and Drive	AIG	Low
TBC-8	Kafka plain listener without authentication (any pod can produce and consume): SCRAM and authorisation on the TLS listener, one KafkaUser per service with its password in Vault, then remove the plain listener (section 8.1)	Platform team	Medium

12.3 Temporary Settings to Revert

Each setting is marked TEMPORARY in the repository. Revert it in the commit that follows the event in the second column.

Table 60 – Temporary settings

Setting	Revert when	Target
Images from cr.terradue.com (cr-terradue-pull in processing, mate-api)	Images copied to Harbor (after the StorageGRID upgrade, TBC-1)	Harbor pull secret
Argo Workflows authModes server + client	Keycloak client argo-workflows	sso + client (.disabled files)
Kafka plain listener without authentication (port 9092)	KafkaUsers and clients on SCRAM over TLS (TBC-8)	TLS listener with SCRAM only

© Terradue 2026 - The copyright in this document is vested in Terradue. This document may only be reproduced in whole or in part, or stored in a retrieval system, or transmitted in any form, or by any means electronic, mechanical, photocopying or otherwise, either with the prior permission of Terradue or in accordance with the terms of ESA Contract No. 4000144570/24/I-KE.

13 Support and Contacts

For assistance, contact Terradue technical support:

- **Email**: copernicuslac-support@terradue.com

© Terradue 2026 - The copyright in this document is vested in Terradue. This document may only be reproduced in whole or in part, or stored in a retrieval system, or transmitted in any form, or by any means electronic, mechanical, photocopying or otherwise, either with the prior permission of Terradue or in accordance with the terms of ESA Contract No. 4000144570/24/I-KE.

Annex A – Public Endpoints

Table 61 – Public endpoints

Host (*.copernicuslac.aig.gob.pa)	Service	Namespace	Certificate	Protection	Status
iam	Keycloak (realm CopernicusLAC-Platform)	iam	iam-tls	/admin and /realms/master → 404	Live
processing	Processing API	mate-api	processing-tls	OPA	Live
discover	STAC API (catalog /main)	catalog	discover-tls	OPA + oauth2-proxy	Live
git	GitLab	gitlab-system	gitlab-tls	GitLab login, Keycloak SSO	Live
cr	Harbor	harbor-system	harbor-tls	Harbor login	Live
view	TiTiler	catalog	view-tls	OPA + oauth2-proxy	To deploy
dashboards	Geobrowser	geobrowser	to create	Keycloak in the browser	To deploy
results, download	Results catalogs, downloads	—	—	OPA + oauth2-proxy	To deploy
www, apps, hub, drive	Portal, App Hub, Drive	—	—	Keycloak	To deploy

© Terradue 2026 - The copyright in this document is vested in Terradue. This document may only be reproduced in whole or in part, or stored in a retrieval system, or transmitted in any form, or by any means electronic, mechanical, photocopying or otherwise, either with the prior permission of Terradue or in accordance with the terms of ESA Contract No. 4000144570/24/I-KE.

Annex B – Changes with Respect to Issue 2.1

Table 62 – Main changes

Issue 2.1	Issue 2.2	Reason
Describes the cluster observed in September 2026 and a target	Describes the platform rebuilt in October 2026; every procedure executed	Handover to operations
Tier 1: Argo CD and GitLab with Helm; Tier 2: Crossplane as Argo CD Applications	Tier 1: Argo CD (helmfile), cert-manager, Contour, Valkey ×2, GitLab, Harbor as Argo CD Applications with manual sync; Tier 2: Crossplane with helmfile only	Blast radius, no circular dependency
platform/ grouped by function, one Application	Layers 10–70, one Application per layer, sync-wave = layer number	Ordering, isolation
Releases in namespace default	Namespace platform	Single place for Tier 3
Secrets of the bootstrap created by hand	SOPS + age in Git, delivery of keys out of band	Reproducible bootstrap
Registry cr.copernicuslac with unresolvable packages	Harbor in Tier 1 with a scripted seed and robot accounts; packages and charts served from Harbor	Self-contained platform
Crossplane 2.3.2, provider-kubernetes v1.2.1, provider-helm :test	Crossplane 2.4.2, v1.4.1, provider-helm v1.0.0	Pinned, current
Vault 1 pod, unseal TBC	Vault HA Raft 3 pods, scripted init/unseal/configure, keys in custody	Availability, operability
Keycloak 26.5, superuser DB access	Keycloak 26.8.1, dedicated role, admin not published, realm hardened (OTP)	Security
OPA prepared, not wired	OPA on processing and catalog hosts, oauth2-proxy, route pattern	Security
Knative from manifests	Knative Operator + KnativeServing	GitOps
Kafka on longhorn, topics declared only	Kafka on longhorn-processing, auto-creation for event topics	Capacity, dynamic topics
Argo in processing	Release argo-workflows, API namespace mate-api, archive argowf	Clean naming
STAC services as manual installs	STAC API as Release, pgSTAC 0.9.12 procedure	GitOps
Known issues of the old cluster	Deployment status, open points, temporary settings	Work list

© Terradue 2026 - The copyright in this document is vested in Terradue. This document may only be reproduced in whole or in part, or stored in a retrieval system, or transmitted in any form, or by any means electronic, mechanical, photocopying or otherwise, either with the prior permission of Terradue or in accordance with the terms of ESA Contract No. 4000144570/24/I-KE.

Annex C – Vault Paths and ExternalSecrets

Checklist of the Tier 3 secrets (KV v2 mount secrets/). Every value must exist before its layer is synced, and must be re-entered if Vault is rebuilt from empty storage.

Table 63 – Vault paths

Vault path	Keys	ExternalSecret (namespace)	Used by
iam/keycloak-external-db	password	keycloak-external-db (iam)	Keycloak database
iam/keycloak-admin	username, password	removed after the first start	Bootstrap admin (new install only)
registry/cr-terradue	dockerconfigjson	cr-terradue-pull (processing, mate-api)	TEMPORARY image pulls
processing/argo-artifacts	accesskey, secretkey	argo-artifacts-creds (processing)	Argo artifacts (user argowf)
processing/argo-postgres	username, password	argo-postgres-config (processing)	Argo archive
processing/argo-sso	client-id, client-secret	argo-workflows-sso-secret (.disabled)	Argo SSO (target)
catalog/pgstac	username, password	external-secret-pgstac → pgstac-secret (catalog)	STAC API (TiTiler planned)
catalog/oauth2-proxy	OAUTH2_PROXY_CLIENT_ID, _CLIENT_SECRET, _COOKIE_SECRET	oauth2-proxy-client-secret-main (catalog)	oauth2-proxy of discover
catalog/titiler-s3, titiler-redis, oauth2-proxy-view	—	planned	TiTiler

```
# writing a dockerconfigjson without exposing the password (works from zsh)
bash -c '
read -rp "registry user: " U </dev/tty; read -rsp "password: " P </dev/tty; echo
kubectl create secret docker-registry x --docker-server=cr.terradue.com --docker-username="$U"
\
  --docker-password="$P" --dry-run=client -o jsonpath="{.data.\.dockerconfigjson}" | base64 -d
\
  | vault kv put secrets/registry/cr-terradue dockerconfigjson=-'

# check all paths of a layer
for k in processing/argo-postgres processing/argo-artifacts registry/cr-terradue; do
  printf '%-28s ' $k
  out=$(vault kv get -format=json secrets/$k 2>&1) \
    && echo "$out" | jq -c '.data.data | keys' || echo "MISSING: ${out%%$'\n'*}"
done
```

© Terradue 2026 - The copyright in this document is vested in Terradue. This document may only be reproduced in whole or in part, or stored in a retrieval system, or transmitted in any form, or by any means electronic, mechanical, photocopying or otherwise, either with the prior permission of Terradue or in accordance with the terms of ESA Contract No. 4000144570/24/I-KE.